

UNIT-I

SCRIPTING

> Web page Designing using HTML

> Scripting basics

- Client side and Server side Scripting

> Java Script

> Object

> names

> literals

> operators and expressions

> statements and features

> events

> windows

> documents

> frames

> data types

> built-in functions

> Browser object model

> Verifying forms

> HTML 5, CSS3, HTML5 Canvas

> Web site creation using tools.

Part-I : Scripting

⇒ Web page Designing using HTML:-

* Creating an HTML document:

> The first step of creating a web page is to create an HTML document.

> HTML is known as Hyper Text Markup Language, an HTML document can be created in any text editor even on a notepad.

Steps:

Step-①:- Open your text editor such as Notepad, Sublimetext, etc.

Step-②:- Write the code given below in the text editor.

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title> first HTML file </title>
```

```
</head>
```

```
<body>
```

```
Hello Everyone !!
```

```
</body>
```

```
</html>
```

Step-③:- Save this file with the .html/.htm extension

Step-④:- Open that file with any browser. The Output will be displayed.

→ Designing web page:-

1. `<h1>...</h1>` to `<h5>...</h5>`:-

* These tags are called header tags and they are used to give heading to your webpage of different sizes.

* `<h1>...</h1>` being the largest Heading to `<h5>...</h5>` being the smallest Heading.

2. Bold Tag (`<strong>...</strong>` (or) `<b>...</b>`):-

* These tags are used to make text look bold.

3. Italic Tag (`<i>...</i>` (or) `<em>...</em>`):-

* These tags are used to make text look italics.

* The only difference between `<i>` and `<em>` is that `<em>` semantically emphasises on important text (or) word whereas `<i>` tag is used to make text just used to make text italics.

4. Ordered List (`<ol>...</ol>`):-

* The HTML `<ol>` tag defines an ordered list.

* An ordered list can be numerical (or) alphabetical. An ordered list starts and ends with a `<ol>` tag.

5. Unordered List (`<ul>...</ul>`):-

* It displays elements in bulletin form.

* An Unordered list starts with `<ul>` tag and each item starts with `<li>` tag.

6. Image Tag:-

* If we need to add an image to our website we need to use the following syntax.

* `<img src = "filename" alt = "name / bit about image" />`

7. Anchor Tag :-

- * This tag is mainly used to connect one webpage to another.

⇒ Scripting basics - client side and Server Side Scripting :-

1> Client side Scripting :-

- * Client side scripting source code is visible to the user.
- * It usually depends on the browser and its version.
- * It runs on the user's computer.
- * It does not provide security for data.
- * In this client side scripting HTML, CSS, and JavaScript are used.
- * No need of interaction with the server.

2> Server side Scripting :-

- * Server side scripting source code is not visible to the user.
- * It does not depend on the client side server.
- * It runs on the web server.
- * It provides more security for data.
- * In this server side scripting PHP, Python, Java, Ruby are used.
- * It is all about interacting with the servers.

⇒ JAVASCRIPT OBJECTS :-

* Javascript objects are the most important data type and form the building blocks for modern JavaScript.

* The 'Object' class represents the javascript data types.

Syntax: `ObjectName.methodName()`.

The complete list of Javascript object is listed below :

1. Javascript object properties :- A Javascript property is a member of an object that associates a key with a value.

* Instance properties: An instance property is a property that has a new copy for every new instance of the class.

2. Javascript object Methods :- Javascript methods are actions that can be performed on objects.

→ Static Methods: If the method is called using the object's class itself then it is called a static method of object.

* Instance Methods: If the method is called on an instance of a class object then it is called an instance method.

⇒ JAVASCRIPT VARIABLE NAMES :-

- * Javascript Variable are the building blocks of any programming language.
- * In Java script, Variables can be used to store reusable values.
- * Javascript identifiers :-
Javascript variables must have unique names, These names are called identifiers.
- * There are some basic rules to declare a variable in Javascript :-
 - * These are case-sensitive
 - * Can only begin with a letter, underscore (" _ ") or "\$" symbol.
 - * It can contain letters, numbers, underscore (or) "\$" symbol.
 - * A Variable name cannot be a reserved keyword.
- * We can declare variables in Javascript in 3 ways:
 - i. Javascript var keyword.
 - ii. Javascript let keyword.
 - iii. Javascript const keyword.

i. Javascript Var :-

- * Javascript var is a keyword used to declare variables in Javascript that are function scoped.
 - * Before the introduction of ES6 all the keywords in Javascript were declared with only "Var" keyword.
- Syntax: `var VariableName = ValueOf var;`

ii. Javascript let :-

* Javascript let is a keyword used to declare variables in javascript that are block scoped.

* Two new keywords were added in the ES6 (ES2015) version of javascript.

Syntax:- let variable-name = value;

iii. Javascript const :-

* ES2015 (ES6) introduced the const keyword to define a new variable.

* The different in const variable declaration to others is that it cannot be re-assigned.

Syntax: const const-name;

const x;

⇒ JAVASCRIPT LITERALS :-

* Literals are the way you represent values in javascript.

* These are fixed values that you literally provide in your application. Source, and are not variables.

Examples:

1. 42

2. 3.14159

3. "To be (or) not to be".

* Integers :-

* Integers can be expressed in decimal, hexadecimal (or) octal format.

* A decimal integer literal consists of a sequence of digits without a leading 0.

* Floating point literals:-

- * A Floating point literal can have the following parts: a decimal integer, a decimal point ("."), a fraction, an exponent, and a type suffix.
- * The Exponent part is an "e" (or) "E" followed by an integer, which can be signed.

examples: 1) 3.1415

2) -3.1E12

3) .1e12

4) 2E-12.

* Boolean Literals:-

- * The boolean type has two literal values: true and false.

* String Literals:-

- * A String literal is zero (or) more characters enclosed in double ("") (or) single (') quotes.
- * A String must be delimited by quotes of the same type; that is, either both single quotes (or) double quotes.

examples:-

1) "blah"

2) 'blah'

3) "1234"

4) "one line\nanother line"

⇒ JAVASCRIPT OPERATORS:-

① Arithmetic Operators:- These are the Operators that operate upon the numerical values and return a numerical value.

> Addition (+):- Addition '+' operator performs addition on two operands, this '+' operator can also be used to concatenate strings.

Ex: $y = 5 + 5$ gives $y = 10$.

> Subtraction (-):- Subtraction '-' operator performs subtraction on two operands.

Ex:- $y = 5 - 3$ gives $y = 2$.

> Multiplication (*):- Multiplication '*' operator performs multiplication on two operands.

Ex:- $y = 5 * 5$ gives $y = 25$.

> Division (/):- Division '/' operator performs division on two operands.

Ex:- $y = 5 / 5$ gives $y = 1$.

> Modulus (%):- Modulus '%' operator gives a remainder of an integer division.

Ex:- $A \% B$ means remainder (A/B)

$y = 5 \% 4$ gives $y = 1$.

> Exponentiation (**):- Exponentiation '**' operator give the power of the first operator raised to the second operator.

Ex:-

$y = 5 ** 3$ gives $y = 125$

> Increment (++):- Increment '+' operator increases an integer value by one.

Ex:- Let $A=10$ and $Y=A++$ then $A=11$, $Y=10$.

if $A=10$ and $Y=++A$ then $A=11$, $Y=11$.

> Decrement(--):- Decrement '-' operator decreases an integer value by one.

Ex:- Let $A=10$ and $Y=A--$ then $A=9$, $Y=9$.

if $A=10$ and $Y=--A$ then $A=9$, $Y=9$.

> Unary(+):- Unary '+' is the fastest and preferred way of converting something into a number.

Ex:- $+a$ means "a" is a positive number.

> Negation(-):- Negation '-' operator gives the negation of an operand.

Ex:- $-a$ means "a" is a negative number.

②. Assignment Operators:- The Assignment operator assigns the right operand value to the left operand.

> Addition Assignment(+=):- Sums up left and right operand values and then assigns the result to the left operand.

Ex:- $Y+=1$ gives $Y=Y+1$.

> Subtraction Assignment(-=):- It subtracts the right side value from the left side value and then assigns the result to the left operand.

Ex:- $Y-=1$ gives $Y=Y-1$.

> Multiplication Assignment ($* =$) :- It multiplies a variable by the value of the right operand and assigns the result to the variable.

Ex:- $Y * = A$ is equivalent to $Y = Y * A$.

> Division Assignment ($/ =$) :- It divides a variable by the value of the right operand and assigns the result to the variable.

Ex:- $Y / = A$ is equivalent to $Y = Y / A$.

> Modulus / Remainder Assignment ($\% =$) :- It divides a variable by the value of the right operand and assigns the remainder to the variable.

Ex:- $Y \% = A$ is equivalent to $Y = Y \% A$.

> Exponentiation Assignment ($** =$) :- This raises the value of a variable to the power of the right operand.

Ex:- $Y ** = A$ is equivalent to $Y = Y ** A$.

> Left Shift Assignment ($<< =$) :- It moves the specified amount of bits to the left and assigns the result to the variable.

Ex:- $Y << = A$ is equivalent to $Y = Y << A$.

> Right Shift Assignment ($>> =$) :- It moves the specified amount of bits to the right and assigns the result to the variable.

Ex:- $Y >> = A$ is equivalent to $Y = Y >> A$.

③ Comparison Operators :- Comparison Operators are mainly used to perform the logical operations that determine the equality (or) different between the values.

> Equality ($==$) :- Compares the equality of two operands, if equal then the condition is true otherwise false.

Ex:- $y=5$ and $x=6$
 $y==x$ is false.

> Strict equality ($===$) :- Compares the equality of two operands with type, if both value and type are equal then the condition is true otherwise false.

Ex:- $y=5$ and $x='5'$
 $y===x$ is false.

> Inequality ($!=$) :- Compares inequality of two operands, True if operands are not equal.

Ex:- Let $x=10$ then $x!=11$ is true.

> Strict inequality ($!==$) :- Compares the inequality of two operands with type, if both value and type are equal then the condition is true otherwise false.

Ex:- Let $x=10$ then $x!== '10'$ is true.

④ Logical Operators :- These operators are used to determine the logic between variables (or) values.

> Logical AND ($\&\&$) :- It checks whether two operands are non-zero, if yes then return the last operand when evaluating from left to right.

Ex:- $y=5$ and $x=6$
 $y\&\&x$ is 6

> Logical OR($\parallel$):- It checks whether two operands are non-zero, if yes then return the first operand when evaluating from left to right

Ex:- $Y=5$ and $X=0$

$Y \parallel X$ is 5

> Logical NOT(!):- It reverses the boolean result of the operand.

Ex:- $Y=5$ and $X=0$.

$!(Y \parallel X)$ is false.

⑤. Bitwise Operator:- The bitwise operator in Javascript is used to convert the number to a 32-bit binary number and perform the bitwise operation.

> Bitwise AND($\&$):- The operator returns true only if both the operands are true.

Ex:- $A=6$, $B=1$

$A \& B = 0$

> Bitwise OR($\parallel$):- The Operator returns true even if one of the operands is true.

Ex:- $A=6$, $B=1$

$A \parallel B = 7$

> Bitwise XOR($\wedge$):- The Operator returns true if both operators are distinct.

Ex: $A=6$, $B=1$

$A \wedge B = 7$

> Bitwise NOT (~): This operator is used to invert the boolean value of the operand.

Ex:- $A = 6$

$\sim A = -7$

> Bitwise Left Shift (<<):- In this two operators are used where the first operand is the number and the second operand is the number of bits to shift to the left.

Ex:- $A = 6, B = 1$

$A \ll B = 12$

> Bitwise Right Shift (>>):- In this two operators are used where the first operand is the number and the second operand is the number of bits to shift to the right.

Ex:- $A = 6, B = 1$

$A \gg B = 3$

⇒ JAVASCRIPT EXPRESSIONS:-

* Javascript expression is a valid set of literals, variables, operators, and expressions that evaluate a single value that is an expression.

* This single value can be a number, a string, (or) a logical value depending on the expression.

* Javascript primary Expressions:

i) this keyword :- that defines the current line of code's execution context.

ii) Await function :- checks that we are not breaking the execution thread.

iii) Grouping operator :- the grouping operator consists of a pair of parenthesis around an expression.

iv) function Expression:- define a function inside an expression.

v) Class expression:- The class name is used internally, but not outside of the class.

⇒ JAVASCRIPT STATEMENTS:-

* The Programming instructions written in a program in a programming language are known as statements.

i. Semicolons :-

* Semicolons separate javascript statements

* A semicolon marks the end of a statement in javascript.

ii. Code Blocks :-

* Javascript statements can be grouped together inside curly brackets, such groups are known as code blocks.

* The purpose of grouping is to define statements to be executed together.

iii. White space :-

* Javascript ignores multiple white spaces.

iv. Line length and line Breaks :-

* Javascript code's preferred line length by most programmers is upto 80 characters.

* The Best place to break a code line in javascript, if it doesn't fit, is after an operator.

✓. Keywords :-

* Keywords are reserved words and cannot be used as a variable name.

* A Javascript keyword tells about what kind of operation it will perform.

* Some commonly used keywords are :-

→ break and continue :- break keyword is used to terminate a loop and continue is used to skip a particular iteration in a loop and move to the next iteration.

→ do.... while :- In this, the statements written within the do block are executed till the condition in while is true.

→ for :- It helps in executing a block of statements till the condition is true.

→ Switch :- This helps in executing a block of codes depending on different cases.

⇒ JAVASCRIPT FEATURES :-

* Javascript is one of the most popular programming languages which includes numerous features when it comes to web development.

1. Lightweight Scripting Language :-

* Javascript is a lightweight scripting language because it is made for data handling in the browser (or) the client side.

* Because javascript is meant for client-side execution for web applications, hence the lightweight nature of javascript is a great feature.

2. Dynamic Typing :-

* Javascript supports dynamic typing which means types of the variable are defined based on the stored value.

* If you declare a variable x then you can store either a string (or) a number type value in javascript (or) an array (or) an object, is known as dynamic typing.

3. Object-oriented programming Support :-

* Starting from ES6, the concept of class and Oop is better defined.

* Also, in javascript, two important principles with Oop in javascript are:

i. Object creation patterns.

ii. Code Reuse patterns.

4. Prototype-based language :-

* Javascript is a prototype-based scripting language.

* This means javascript uses prototypes instead of classes (or) inheritance.

⇒ JAVASCRIPT EVENTS :-

* Javascripts has events to provide a dynamic interface to a webpage.

* These events are hooked to elements in the Document object Mode (DOM).

* There are some javascript events:-

1. Javascript onclick events: This is a mouse event and provokes any logic defined if the user clicks on the element it is bound to.
2. Javascript onkeyup event: This event is a keyboard event and executes instructions whenever a key is released after pressing.
3. Onmouseover event: This event corresponds to hovering the mouse pointer over the element and its children, to which it is bound to.
4. Javascript onmouseout event: whenever the mouse cursor leaves the element which handles a mouseout event, a function associated with it is executed.
5. Javascript onchange event: This event detects the change in value of any elements 'listening' to this event.
6. Javascript onload event: When an element is loaded completely, this event is evoked.
7. Javascript onfocus event: An element 'listening' to this event executes instructions whenever it receives focus.
8. Javascript onblur event: This event is evoked when an element loses focus.

⇒ WINDOWS:-

- * It represents the browser window in which you are seeing the content.
- * The properties related to it are stored in the window object.
- * It is loaded before the document because container document.
- * The Window is the global elements for an object, functions, etc...

Syntax: window object:
window.propertyname;

Example:

window.document.title will return the title of the document.

⇒ DOCUMENTS:

- * It represents the document loaded inside the window (or) browser.
- * The properties related to it are stored in the document object.
- * It is loaded after the loading window because window contains a document.
- * It is the root element of the document object model.

Syntax : document object :
document.propertyname.

Example : document.title will return the title of the document.

⇒ HTML / <frame> Tag :-

* HTML Frames are used to divide the web browser window into multiple sections where each section can be loaded separately.

* A Frameset tag is the collection of frames in the browser window.

→ * Creating Frames :-

* Instead of using body tag, use frameset tag in HTML to use frames in web browser.

* But this Tag is deprecated in HTML 5.

* The Frameset tag is used to define how to divide the browser.

* Each frame is indicated by frame tag and it basically defines which HTML documents shall open into the frame.

→ * Attributes of frameset tag :-

→ cols :

* The cols attribute is used to create vertical frames in web browser.

* This attribute is basically used to define the no. of columns and its size inside the frameset tag.

Example :-

```
<frameset cols = "300, 400, 300">
```

→ rows :-

- * The rows attribute is used to create horizontal frames in web browser.

- * This attribute is used to define no. of rows and its size inside the frameset tag.

Example :-

```
<frameset rows = "300, 400, 300">
```

→ border :-

- * This attribute of frameset tag defines the width of border of each frames in pixels.

- * Zero value is used for no border.

Example :-

```
<frameset border = "4" frameset>
```

→ frameborder :-

- * This attribute of frameset tag is used to specify whether the three-dimensional border should be displayed between the frames (or) not for this use two values 0 and 1, where 0 defines for no border and value 1 signifies for yes there will be border.

⇒ DATA TYPES :-

① Primitive Data Types:-

* The predefined data types provided by javascript language are known as primitive data types.

> Number :- Javascript numbers are always stored in double-precision 64-bit binary format. IEEE 754.

Ex:- `let num = 2;`

`let num2 = 1.3;`

`let num3 = Infinity;`

`let num4 = 'something here too' / 2;`

> String :- Javascript strings are similar to sentences, they are made up of a list of characters, which is essentially just an "array of characters, like"

Hello GeeksforGeeks etc...

Ex:- `let str = "Hello There";`

`let str2 = 'single quotes works fine';`

`let phrase = 'can embed $ {str}';`

> Boolean :- Represent a logical entity and can have two values : true (or) false.

Ex: `let is coding = true;`

`let isold = false;`

> Null :- This type has only one value that is null.

Ex:- `let age = null;`

> undefined :- A variable that has not been assigned a value is undefined

Ex: `let x;`

`console.log(x);`

②. Non-primitive Data Types:

* The Data types that are derived from primitive data types that are of the javascript language are known as non-primitive data types.

> object: It is the most important data type and forms the building blocks for modern javascript.

Ex:- let person = new Object();

let person = { };

⇒ BUILT-IN- FUNCTIONS:-

1. eval:-

* eval evaluates a string and returns a value.

* Expression is evaluated, and if the result is not a string, the result is returned, if the result is a string, it is taken to be a javascript program, and it is evaluated.

Ex: Var x = 2

Var y = 39

Var z = "42"

println (eval ("x+y+1"))

println (eval (z))

2. parseInt:

* Parses a string argument and returns an Integer of the specified radix (or) base

Syntax: parseInt (string)

parseInt (string, radix)

Ex:-
parseInt ("F", 16)
parseInt ("17", 8)
parseInt ("15", 10)
parseInt (* 15.99, 10)
parseInt ("Fxx123", 16)
parseInt ("1111", 2)
parseInt ("15 * 3", 10)

3. parseFloat:-

* Parses a string argument and returns a floating point number.

Syntax: parseFloat (string)

Ex:-
parseFloat ("3.14")
parseFloat ("314e-2")
parseFloat ("0.0314E+2")
Var x = "3.14"
parseFloat(x).

4. escape:-

* Returns the hexadecimal encoding of an argument in the ISO Latin-1 character set.

Syntax: escape ("string")

ex: escape ("abc&%")

5. Unescape:-

* Returns the ASCII string for the specified value.

Syntax: Unescape ("string")

Ex: Unescape ("%26").

⇒ BROWSER OBJECT MODEL (BOM) :-

* The Browser Object Model (BOM) is a browser specific convention referring to all the objects exposed by the web browser.

* The BOM allows javascript to "interact with" the browser.

Properties:

> Screen.width : The Screen.width property returns the users screen width in pixels.

Ex: `<p id = "GFG"></p>`

`<script>`

`document.getElementById("GFG").innerHTML =`

`"The screen width is" + Screen.width;`

`</script>`

> Screen.height : The Screen.height property returns the users screen height in pixels.

Ex:

`<p id = "GFG"></p>`

`<script>`

`document.getElementById("GFG").innerHTML =`

`"The screen height is" + screen.height;`

`</script>`

> screen.availwidth : The Screen.availwidth property returns the users screen width in pixels, excluding the interface features.

Ex: `<p id = "GFG"></p>`

`<script>`

`document.getElementById("GFG").innerHTML =`

"The availability screen width is" + screen.availwidth;
</script>

> screen.availHeight: The screen.availHeight property returns the users screen height in pixels excluding the interface features.

Ex:

```
<p id = "GFG"></p>
```

```
<script>
```

```
document.getElementById("GFG").innerHTML =
```

```
"The availability screen height is" + screen.availHeight;
```

```
</script>
```

> Screen.ColorDepth: The screen.ColorDepth property returns the bits to be used to display one color.

Ex:

```
<p id = "GFG"></p>
```

```
<script>
```

```
document.getElementById("GFG").innerHTML =
```

```
"The screen color depth is" + screen.ColorDepth;
```

```
</script>
```

> Screen.pixelDepth: The screen.pixelDepth property returns the pixel depth of the screen.

Ex:

```
<p id = "GFG"></p>
```

```
<script>
```

```
document.getElementById("GFG").innerHTML =
```

```
"The screen pixel depth is" + screen.pixelDepth;
```

```
</script>
```

⇒ VERIFYING FORMS:

* Forms are used on webpages for the user to enter their required details that further send to the server for processing.

* A form is also known as a web form (or) HTML form.

Prerequisites:

> HTML: HTML is used to create the form.

> CSS: CSS to design the layout of the form.

> Javascript: Javascript to validate the form.

Validating a form: The data entered into a form needs to be in the right format and certain fields needs to be filled in order to effectively use the submitted form.

Validations used in below code snippet:

// Javascript regex for Email Validation

```
var reg Email = /^[1-9]?[a-zA-Z0-9]{1,30}@([1-9]?[a-zA-Z0-9]{1,30})+$/;
```

```
var reg phone = /^[0-9]{10}$/;
```

```
var reg Name = /^[a-zA-Z]{1,30}$/;
```

⇒ HTML 5:

* HTML stands for Hyper Text Markup Language.

* It is used to design web pages using a markup language.

Features:

* It has introduced new multimedia features which supports both audio and video controls by using

<audio> and <video> tags

- * There are new graphics elements including vector, graphics and tags.

Removed Elements from HTML 5 :-

1. <acronym>
2. <applet>
3. <basefont>
4. <big>
5. <centre>
6. <dir>
7.
8. <frame>
9. <frameset>
10. <object>

New Added Elements in HTML 5 :-

1. <article> : The <article> tag is used to represent an article, the content within the <article> tag is independent from the other content of the site.
2. <aside> : The <aside> tag is used to describe the main object of the web page in a shorter way like a highlighter.
3. <figcaption> : The <figcaption> tag in HTML is used to set a caption to the figure element in a document.
4. <figure> : The <figure> tag in HTML is used to add self-contained content like illustrations, diagrams, photos (or) codes listing in a document.
5. <header> : It contains the section heading as well as other content, such as a navigation links, table of contents, etc. ...

=> CSS 3 :-

- * CSS 3 stands for cascading style sheet level 3 which is the advanced version of CSS.
- * It is used for structuring, styling and formatting web pages.
- * Several new features have been added to CSS 3 and it is supported by all modern web browsers.
- * The Most important feature of CSS 3 is the splitting of CSS standards into separate modules that are simpler to learn and use.

New features of CSS 3 :-

Combinator : CSS 3 has a new General sibling Combinator which matches up with sibling elements via the tilde (~) combinator.

CSS Selectors : CSS 3 selectors are much advanced in comparison to simple selectors offered by CSS, and are termed as a sequence of easy to use and simple selectors.

Pseudo-elements : plenty of new pseudo-elements have been added to CSS 3 to give easy styling in depth.

Border style : The Latest CSS 3 also has new border styling features like border-radius, image-slice, image-source, and values for "width stretch" etc....

Background style properties: New features like background-clip, size, style, and origin properties have been added to CSS3.

* ⇒ HTML CANVAS :-

* The HTML "Canvas" element is used to draw graphics via javascript.

* The "Canvas" element is only a container for graphics.

* One must use javascript to actually draw the graphics.

* Canvas has several methods for drawing paths, boxes, circles, text, and adding images.

Syntax :

```
<canvas>  
  Content ....  
</canvas>
```

HTML Canvas Methods :

setInterval (callback, time): This method repeatedly executes the supplied code after a given time.

setTimeout (callback, time): This method executes the supplied code only once after a given time.

⇒ WEB SITE CREATION USING TOOLS :-

* Web development refers to the creating, building, and maintaining of websites.

* It includes aspects such as web design, web publishing, web programming, and data-base management.

The word web Development is made up of two words, that is:

web: It refers to websites, web pages (or) anything that works over the internet.

Development: It refers to building the application from scratch.

web Development can be classified into two ways:

1. Frontend Development:

- * The Part of a website where the user interacts directly is termed as frontend.

- * It is also referred to as the 'client side' of the application.

Popular Frontend Technologies:

HTML: HTML stands for Hyper Text Markup Language, it is used to design the frontend portion of web pages using markup language.

CSS: Cascading Style Sheets fondly referred to as CSS is a simply designed language intended to simplify the process of making web pages presentable.

Javascript: Javascript is a scripting language used to provide a dynamic behaviour to our website.

Bootstrap: Bootstrap is a free and open-source tool collection for creating responsive websites and web applications.

2. Backend Development:

* Backend is the server side of a website.

* It is a part of the website that users cannot see and interact with.

popular Backend Technologies:

PHP: PHP is a server-side scripting language designed specifically for web development.

Java: Java is one of the most popular and widely used programming languages.

Python: Python is a programming language that lets you work quickly and integrate systems more efficiently.

Node.js: Node.js is an open source and cross-platform runtime environment for executing javascript code outside a browser.

UNIT-II

JAVA

- Introduction to Object-Oriented programming
- Features of Java.
- Data types.
- Variables and arrays.
- Operators.
- Control statements
- Classes and Methods
- Inheritance
- Packages and Interfaces.
- Exception Handling
- Multithreaded programming
- Input/output
- Files
- Utility classes
- String Handling.

⇒ INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING

1. OOPS [Object Oriented Programming] is a core of java programming, which is used for designing a program using classes and objects.

OOPS Concepts:

1. class
2. Objects
3. Data Abstraction
4. Encapsulation
5. Inheritance
6. polymorphism.

1. Class: A class is a user-defined data type, it consists of data members and member functions, which can be accessed and used by creating an instance of that class.

2. objects: An object is an instance of a class, when a class is defined, no memory is allocated but when it is instantiated (i.e., an object is created) memory is allocated.

3. Data Abstraction: Data Abstraction refers to providing only essential information about the background details (or) implementation.

4. Encapsulation: Encapsulation is defined as the wrapping of data and methods into a single unit is known as encapsulation and it is also known as data-hiding.

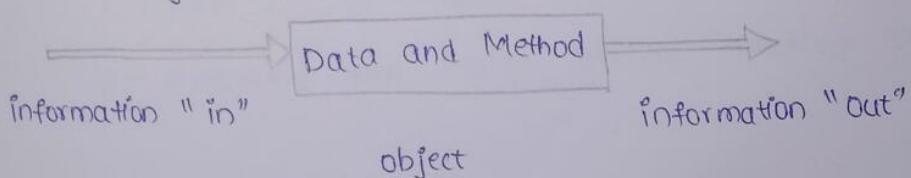
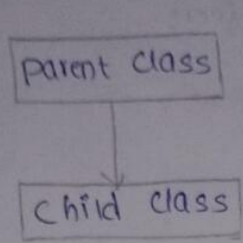
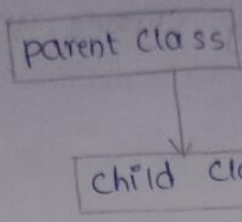


Fig: Encapsulation

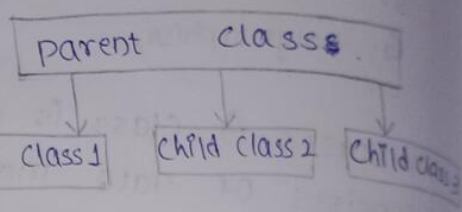
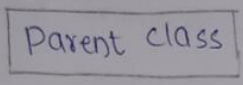
5. Inheritance: The capability of a class to inherit properties and characteristics from another class is called inheritance.



Single inheritance

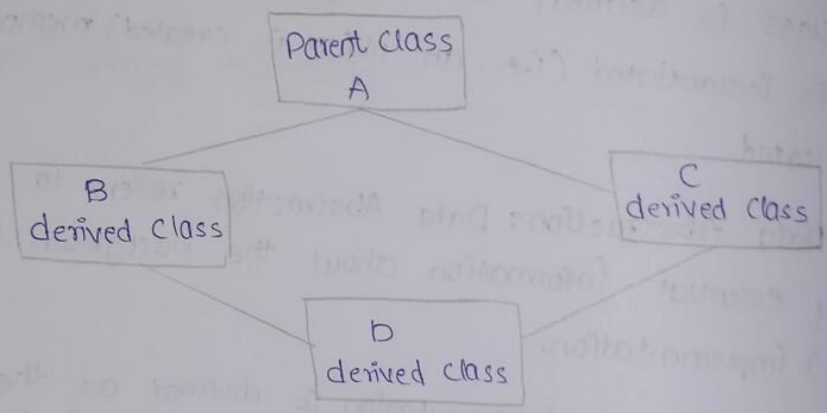


Multiple inheritance.



Hierarchical inheritance

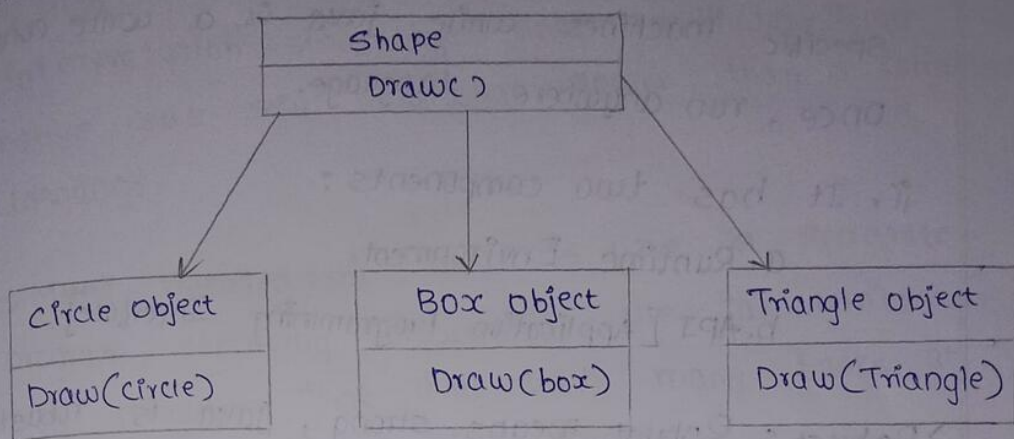
Multilevel inheritance



Hybrid class

6. Polymorphism: Polymorphism is defined as the ability of a message to be displayed in more than one form.

example:



⇒ FEATURES OF JAVA :-

* The Primary objective of Java Programming language creation was to make it portable, simple and Secure programming language.

* There are mainly 12 types of features of Java, They are:

> Simple: Java language is simple because:

- i. Syntax is based on C++
- ii. Removed many confusing and/or rarely-used features.

> Object-Oriented: OOPS (Object Oriented Programming) is a core of java programming, which is used for designing a program using classes and objects.

> Secured: Java is secured because:

- i. There is no explicit pointers
- ii. Programs run inside virtual machine sandbox.

> Platform Independent: Java is platform independent because:

i. It is different from other languages like C++ etc... which are compiled into platform specific machines while Java is a write once, run anywhere language.

ii. It has two components:

a. Runtime Environment

b. API [Application Programming Interface].

> Robust: Robust means strong, Java is robust because:

a. It uses strong memory management.

b. There is a lack of pointers that avoids security problems.

> Portable: We may carry the Java bytecode to any platform.

> Architecture-neutral: There are no implementation dependent features.

Ex: Size of primitive types is set.

> Dynamic: Java is a dynamic loading of classes language because:

i. It supports the dynamic loading of classes.

ii. It also supports functions from its native languages, i.e., C and C++.

> Interpreted: Java byte code is translated on the fly to native machine instructions and is not stored anywhere.

> High Performance: Java is faster than traditional interpretation. Since byte code is "close" to native code still somewhat slower than a compiled language.

> Multi-threaded: A Thread is like a separate program, executing concurrently, we can write java programs that deal with many tasks at once by defining multiple threads.

> Distributed: Java is distributed because:

i. It facilitates users to create distributed applications in java.

ii. RMI and EJB are used for creating distributed applications.

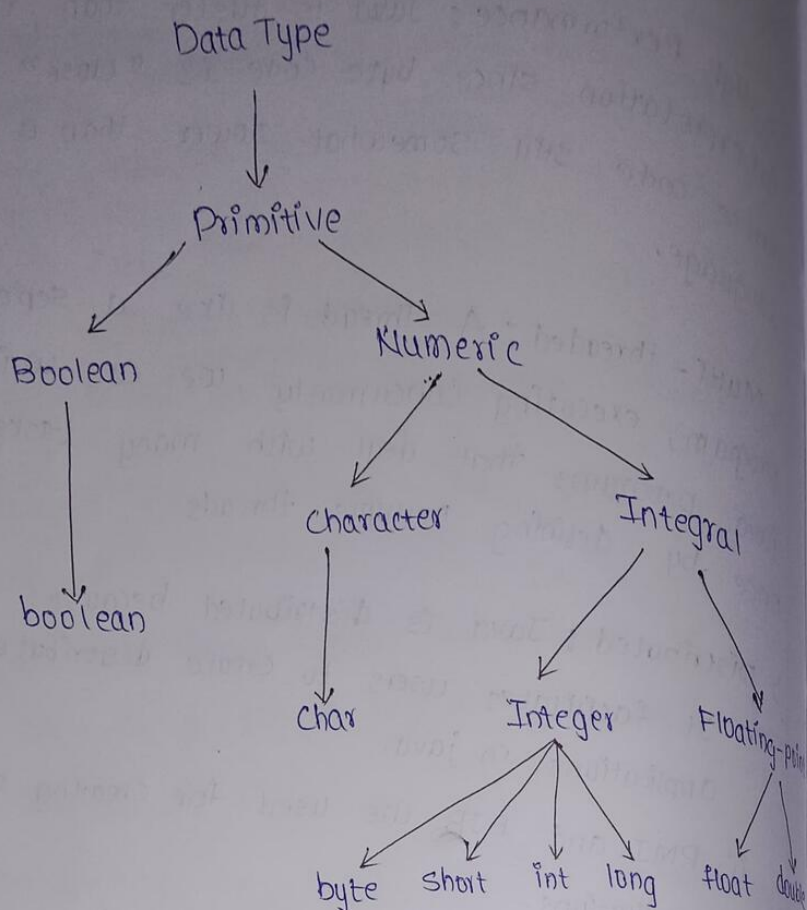
⇒ DATA TYPES:

* Data types in java are of different sizes and values that can be stored in the variable that is made as per convenience and circumstances to cover up all the test cases.

* There are mainly two types of data types.

They are:

① Primitive Data Types: Data Types make a variable to store a single value at a time.

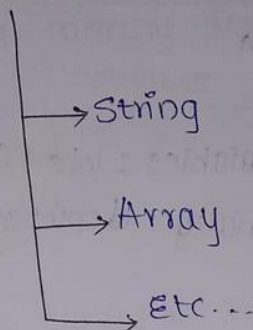


Type	Size (byte)	Range	Default
byte	1	-128 to 127	0
Short	2	-32768 to 32767	0
Int	4	-2147483648 to 2147483647	0
Long	8	-9223372036854775808 to 9223372036854775807	0
Float	4	3.4e38 to 1.4e-45	0.0
Double	8	1.e-308 to 4.9e-324	0.0
Boolean	JVM Specific	true (or) false	FALSE
char	2	0 to 65535	\u0000

②. Non-Primitive Data Types :

Derived data types are those that are made by using any other data type and can make a variable to store multiple values.

Data Type
↓
Non-Primitive



> String: Strings are defined as an array of characters, the string is designed to hold a sequence of characters in a single variable.

Syntax:

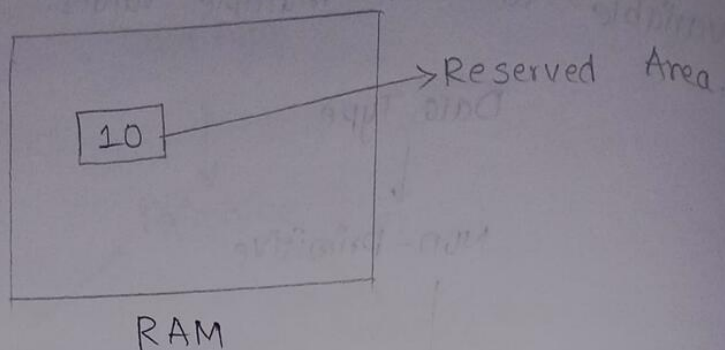
`<string-type> <string-variable> =
" <sequence-of-string> " ;`

> Array:

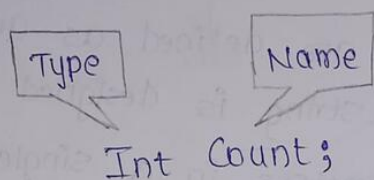
- * An Array is a group of like-typed variables that are referred to by a common name.
- * Arrays in Java work differently than they do in C/C++.

⇒ VARIABLES:

Variable is name of reserved area allocated in memory.



Declaration of Variables: We can declare variables in Java as pictorially depicted below as a visual aid



Data type: Type of data that can be stored in this variable.

Data_name: Name was given to the variable.

Syntax: Datatype Variable_name;

Initialization of Variables: It can be perceived with the help of 3 components that areas follows:

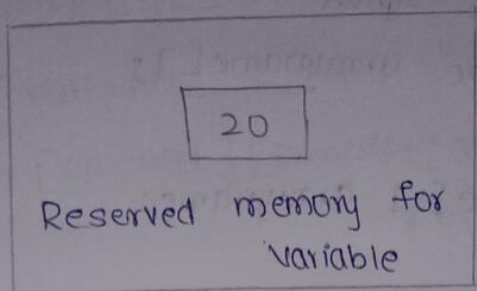
- > Datatype: type of data that can be stored in this variable.
- > Variable-name: Name given to the variable.
- > Value: It is the initial value stored in the variable.

Int age = 20;

↓ ↓ ↓

Data Variable Value

type name



RAM

Types of Variables:

Local Variables: A Variable that is declared inside the method is called Local Variable.

Instance Variable: A Variable that is declared inside the class but outside the method is called Instance variable.

Static Variable: A Variable that is declared as static is called Static Variable.

Example:

class A

{

int data = 50; // instance variable

static int m = 100; // static variable

void method()

{

int n = 90; // local variable

}

} // end of class.

⇒ ARRAYS: Java array is an object the elements of similar data type.

Types of Arrays:

1) Single dimensional Array:

Array Declaration Syntax:

Datatype arrayname[];

or

Datatype[] arrayname;

Array Initialization Syntax:

arrayname[subscript] = Value;

Array Creation Syntax:

arrayname = new datatype[size];

Rules for creating an Array:-

Rule 1: At the time of array creation compulsory we should specify the size otherwise we will get compile time error.

Rule 2: It is legal to have an array with size zero in java.

Rule 3: If we are taking array size with negative int value then we will not get runtime exception saying Negative Array Size Exception.

Rule 4: The Allowed data type to specify array size are byte, short, char, int.

Rule 5: The Maximum allowed array size in java is maximum value of int size [2147483647]

2) Multi dimensional Array :-

Array declaration syntax:

```
datatype arrayname[ ][ ];
```

or

```
datatype [ ][ ] arrayname;
```

Array initialization syntax:

```
datatype [subscript] [subscript] arrayname =  
    { list of values };
```

Array Creation Syntax:

```
arrayname = new datatype [size] [size];
```

Ex: `int a[ ][ ];`

`a = new int[3][2];`

=> OPERATORS :

* An Operator is a symbol which perform some operation.

* There are mainly '8' types of operators. They are:

i. Arithmetic operators.

ii. Relational operators

iii. Logical operators

iv. Assignment operators.

v. Increment and decrement operators.

vi. Conditional operators.

vii. Bitwise operators

viii. Special operators.

i) Arithmetic Operators

* Arithmetic operators are used to perform basic arithmetic operations.

* There are mainly 5 types of Arithmetic operators.

They are:

* $+$: Addition

* $-$: Subtraction

* $*$: Multiplication

* $/$: Integer Division

* $\%$: Modulo Division.

Example: $a=5$, $b=2$

$$a+b = 5+2 = 7$$

$$a-b = 5-2 = 3$$

$$a*b = 5*2 = 10$$

$$a/b = 5/2 = 2$$

$$a\%b = 5\%2 = 1$$

ii) Relational Operators:

* Relational Operators are used to compare two quantities.

* They always give a result of true (or) false.

* There are mainly 'six' types of relational operators. They are:

$<$: less than

$<=$: less than (or) equal to

$>$: greater than

$>=$: greater than (or) equal to

$==$: Equal to

$!=$: Not equal to

Example: $a = 5$, $b = 2$

$a < b \Rightarrow 5 < 2 = \text{False}$

$a > b \Rightarrow 5 > 2 = \text{True}$

$a = b \Rightarrow 5 = 2 = \text{False}$

$a <= b \Rightarrow 5 <= 2 = \text{False}$

$a >= b \Rightarrow 5 >= 2 = \text{True}$

$a != b \Rightarrow 5 != 2 = \text{True}$

(iii) Logical Operators:-

* Logical operators are used to combine two (or) more conditions.

* They also always give a result of True/False.

* There are mainly 3 types of logical operators.

They are:-

$\&\&$: Logical AND.

$||$: Logical OR

$!$: Logical NOT

Example: $a = 5$, $b = 3$, $c = 1$

$(a < b) \&\& (b < c) = \text{False}$

$(a > b) || (b < c) = \text{True}$

$(a < b) ! (b < c) = \text{False}$.

(iv) Assignment Operators:-

* Assignment operators are used to assign the value obtained from right side expression to left side variable.

* There are mainly two types of assignment operators. They are:

$=$: Equal to

$V \text{ op} = e$: Shorthand Assignment Operator.

V : Variable

op : Operator

e : expression.

Example:

$$c = a + b$$

1. $a = a + 5$ can be written as $a += 5$
2. $x = x / y$ can be written as $x /= y$

v. Increment and Decrement Operators:

* There are 2 special operators in web programming language, Increment ($++$) operator and Decrement ($--$) operator.

* The Increment operator increases the value of an operand by 1.

* The Decrement operator decreases the value of an operand by 1.

* They can be used as 'prefix' and 'postfix'.

Example:

$$x = 5;$$

$$++x$$

$$x = 5;$$

$$x++$$

$$x = 5;$$

$$--x$$

$$x = 5;$$

$$x--$$

After execution:

$$x = 6$$

$$x = 6$$

$$x = 4$$

$$x = 4$$

vi. Conditional operators: There are mainly 3 types of Conditional operators. They are:

> **Unary Operators:** The operators which use only one operand to perform the operation are known as Unary Operators.

$$\text{Ex: } (++), (--)$$

> Binary Operators: The Operators which uses two operands to perform the operation are known as Binary Operators.

Ex:- '+', '-', '*', '/', '%'.

> Ternary Operators: The Operators which uses 3 operands to perform the operation are known as Ternary Operators.

Ex:- '?' :

> Bitwise Operators :

vii. Bitwise Operators:

* The Bitwise operators are used to perform operations on bits [Binary digits] i.e 0 (or) 1

* There are mainly six types of Bitwise operators. They are:

& : Bitwise AND

! : Bitwise OR

^ : Bitwise XOR

<< : Leftshift

>> : Rightshift

~ : Complement.

viii. Special operators: There are mainly 3 types of Special operators. They are:

i. size of operators

ii. Membership operators (• →)

iii. Pointer operators (*, &).

⇒ CONTROL STATEMENTS: There are mainly 2 types of control statements. They are:

1. Decision Making In Java: There are mainly 2 types of Decision making statements. They are:

i. If Statement:

* 'If' statement is used to evaluate a condition.

* There are mainly four types of if statement. They are:

> Simple if statement: It evaluates a Boolean expression and enables the program to enter a block of code if the expression evaluates to true.

Syntax: if (condition)

```
{  
    statement 1;  
}
```

> if-else statement: The if-else statement is an extension to the if-statement, which uses another block of code, i.e., else block.

Syntax:

```
if (condition)  
{  
    statement 1;  
}  
else  
{  
    statement 2;  
}
```

> Nested If Statement : The Nested-if statement is the if statement can contain a if (or) if-else statement inside another if (or) else-if statement.

Syntax :

```
if (Condition 1)
```

```
{
```

```
    Statement 1 ;
```

```
    if (Condition 2)
```

```
    {
```

```
        Statement 2 ;
```

```
    }
```

```
    else
```

```
    {
```

```
        Statement 3 ;
```

```
    }
```

```
}
```

> If - else - if : The if - else - if statement contains the if - statement followed by multiple else - if statements.

Syntax : if (Condition 1)

```
{
```

```
    Statement 1 ;
```

```
}
```

```
else if (Condition 2)
```

```
{
```

```
    Statement 2 ;
```

```
}
```

```
else
```

```
{
```

```
    Statement 2 ;
```

```
}
```

1. Switch statement:

- * Case value must be of the same type as expression used in switch statement.
- * Case value must be a constant (or) literal.
- * Case values should be unique.
- * If it is duplicate, then program will give compilation error.

2. Looping statements in Java: There are mainly 4 types of looping statements. They are:-

> while: It checks and evaluates the condition and if it is true then executes the body of loop.

Syntax: Initialization;

while (condition)

{

statement 1;

increment / decrement;

}

> Do...while: It has only one difference that in do...while, condition is checked after the execution of the loop body.

Syntax:

do

{

statement 1;

} while (condition);

> For statement: It is the easiest way to construct a loop structure in code as initialization of a variable, a condition and increment/decrement are declared only in a single line of code.

Syntax: `for (initialization; condition; increment/decrement)`
`{`
 Statement;
`}`

> For-Each loop statement: for-each-loop is used to traverse through elements in an array.

Syntax: `for (data-type var : array-name/collection-name)`
`{`

 // statements

`}`

3. Branching statements in Java: There are mainly two types of branching statements. They are:

> Break: Break statement is used to terminate the execution and bypass the remaining code in loop.

> Continue: Continue statement works same as break but the difference is it only comes out of loop for that iteration and continue to execute the code for next iterations.

⇒ **CLASSES**:- The Java class defines the blue print of an object, every class of Java programming language has the following characteristics.

Identify: It is the name given to the class.

State: Represents data values that are associated with an object.

Behaviour: Represents actions can be performed by an object.

Creating a class: In Java, we use the keyword `class` to create a class, A class in Java contains properties as variables and behaviours as methods.

Syntax: `class <className>`

```
{  
    data members declaration;  
    methods definition;  
}
```

Creating an object: When an object of a class is created, the class is said to be instantiated, all the objects are created using a single class have the same properties and methods.

Syntax:

```
<className> <objectName> = new <className> ();
```

⇒ **Methods:** A Method is a block of statements under a name that gets executed only when it is called, Every method declaration has the following characteristics.

return Type: Specifies the data type of a return value.

name: Specifies a unique name to identify it.

parameters: The data values it may accept (or) receive.

{ }:- Defines the block belongs to the method.

Creating a method:- A Method is created inside the class and it may be created with any access specifier.

Syntax:

Class <ClassName>

{

<Access Specifier> <return Type> <methodName>

{

....

block of statements;

....

}

}

Calling a method: In java, a method call precedes with the object name of the class to which it belongs and a dot operator.

Syntax:

<objectName> <Method Name> (actual Arguments);

Variable arguments of method: In java, a method can be defined with a variable number of arguments, that means creating a method that receives any number of arguments of the same data type.

Syntax:

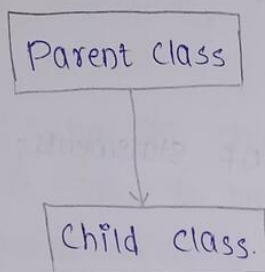
<returnType> <method Name> (data Type Parameter Name);

=> INHERITANCE :- The Capability of a class to derive properties and characteristics from another class is called inheritance.

Types of inheritance:

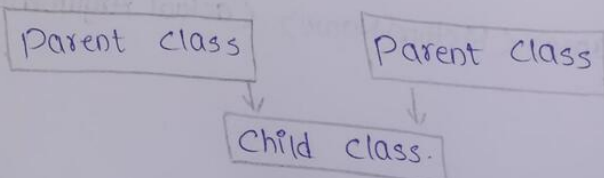
1. Single inheritance: single inheritance refers to a child and parent class relationship where a class extends the another class.

Example:



2. Multiple Inheritance: Multiple inheritance refers to the concept of one class extending more than one classes, which means a child class has two parent classes.

Example:



3. Multilevel inheritance: Multilevel inheritance refers to a child and parent class relationship where a class extends the child class.

Example:

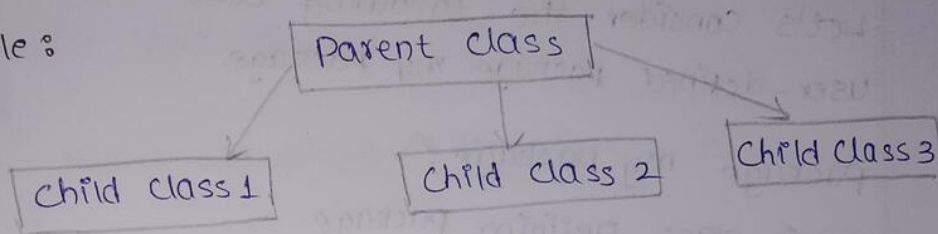
Parent class

derived class 1

derived class 2

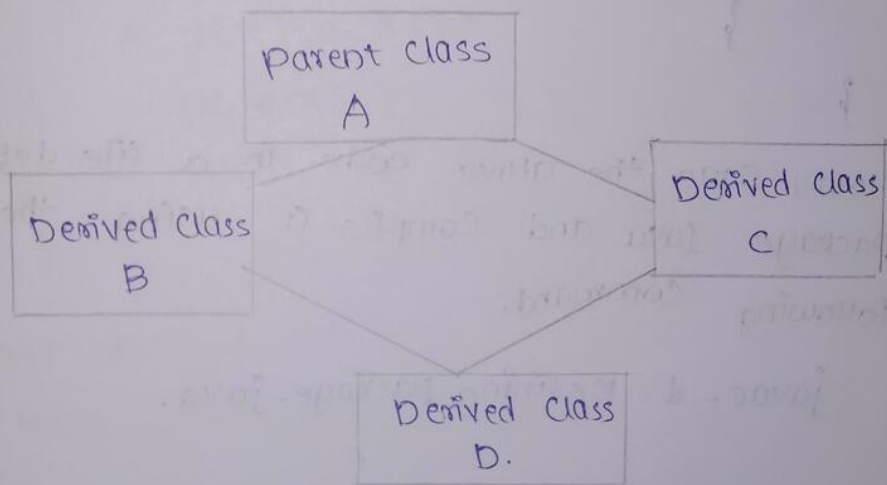
4. Hierarchical inheritance: Hierarchical inheritance refers to a child and parent class relationship where more than one classes extends the same class.

Example:



5. Hybrid inheritance: Combination of more than one type of inheritance in a single program.

Example:



⇒ PACKAGES :-

Defining a package:

* We use the package keyword to create (or) define a package in java programming language.

Syntax:

```
package packageName;
```

* The package statement must be the first statement in the program.

* The package name must be a 'single word'.

* The package name must use 'camel case Notation'.

Let's consider the following code to create a user-defined package my package.

```
package my package;
```

```
public class Defining Package
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        System.out.println("This class belongs to  
        my package");
```

```
    }
```

```
}
```

Now, Save the above code in a file defining package java and compile it using the following command.

```
javac -d. Defining Package.java.
```

Run the program Use the following command.

java my package. Defining package.

Importing packages: There are mainly 3 types of importing packages. They are:

> Using packagename*:-

If you use package*, then all the classes and interfaces of this package will be accessible but not subpackages.

Ex:-

```
package pack;
```

```
public class A
```

```
{
```

```
    public void msgsc()
```

```
    {
```

```
        System.out.println ("Hello");
```

```
    }
```

```
}
```

```
import pack*;
```

```
class B
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        A obj = new A();
```

```
        obj.msg();
```

```
    }
```

```
}
```

Output: 1

Hello.

> Using packagename. classname :-

If you import package. classname then, only declared class of this package will be accessible.

Ex :- package pack;

public class A

{

public void msgsc()

{

System.out.println("Hello");

}

}

import pack.A;

class B

{

public static void main (String args[])

{

A obj = new A();

obj.msgsc();

}

}

Output :

Hello.

> Using fully qualified name:

If you use fully qualified name then, only declared class of this package will be accessible.

* Now there is no need to import.

example :-

```
package pack;
```

```
public class A
```

```
{ public void msgsc()
```

```
{ system.out.println ("Hello");
```

```
}
```

```
}  
class B
```

```
{ public static void main (String args[])
```

```
{ pack.A obj = new pack.A();
```

```
obj.msg();
```

```
}
```

```
}
```

Output :

Hello.

⇒ INTERFACES!

Defining an interface :-

* An interface in java is a "blueprint of a class"

* It is used to achieve and "multiple inheritance"

* It is used to achieve "abstraction"

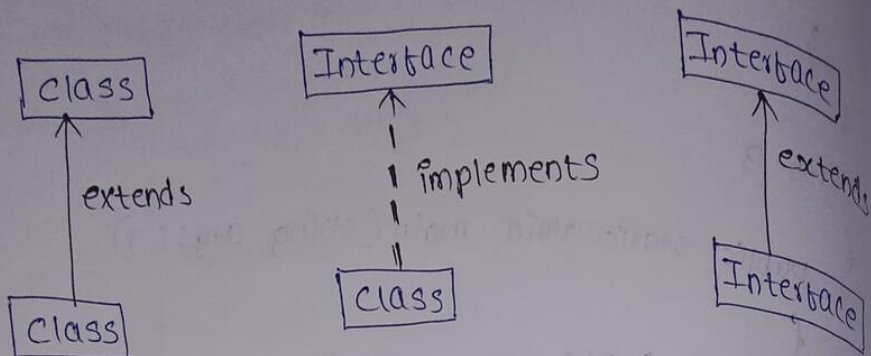
* It is used to achieve "loose coupling"

Types of interfaces:

There are mainly 5 types of interfaces. They are:

> Implementing interfaces:

* As shown in the figure given below, a class extends another class, an interface extends another interface, but a class implements an interface.



> Nested interfaces :-

* An Interface which is declared within another interface (or) class is known as nested interface.

* The Nested interfaces are used to group related interfaces so that they can be easy to maintain.

> Applying interfaces :-

* Interface implements multiple inheritance into the same class.

* A class can extend only class but it can implements multiple interfaces.

> Variables in interfaces :-

* An Interface can also contain variables just as classes but the variables in interface must be of public, static and final access modifiers.

* If the variable are not specified with access specifiers then during compilation process compiler will provide this access specifier.

> Extending Interfaces:-

- * An Interface can extend another interface.
- * An Interface cannot extend multiple interfaces.
- * An Interface can implement neither an interface nor a class.
- * The class that implements child interface needs to provide code for all the methods defined in both child and parent interfaces.

Example program:-

```
interface MyInterface
```

```
{  
    public void method();  
}
```

```
class Demo implements MyInterface
```

```
{  
    public void method()
```

```
{  
        System.out.println("Implementation of method");  
    }
```

```
    public static void main(String arg[])
```

```
{
```

```
        Demo obj = new Demo();
```

```
        obj.method();  
    }
```

```
}
```

Output:

Implementation of method.

⇒ EXCEPTION HANDLING :-

Definition :- The Exception Handling is a mechanism to handle the runtime errors so that the normal flow of the application can be maintained.

* Fundamentals of Exception handling :-

Java exception handling is managed via throw.

5 Keywords. They are:

1. Try: The try block contains a set of statements where an exception can occur.

2. Catch: The catch block is used to handle the uncertain condition of a try block.

3. Throw: The throw keyword is used to transfer control from the try block to the catch block.

4. Throws: The throws keyword is used for exception handling without try and catch block.

5. Finally: It is executed after the catch block.

Types of exception handling :-

1. Checked Exception:

* An Exception that is checked by the compiler at the time of compilation is called a checked exception.

* The following are a few built-in-classes used to handle checked exceptions in java.

i. IO Exception

ii. FileNotFoundException

iii. ClassNotFoundException

iv. SQLException

v. DataAccessException

vi. InstantiationException

viii. UnknownHostException

2. Unchecked Exception :-

* An Exception that cannot be caught by the compiler but occurs at the time of program execution is called an Unchecked Exception.

* The following are few built-in-classes used to handle unchecked exceptions in java.

i. Arithmetic Exception

ii. NullPointerException

iii. NumberFormat Exception

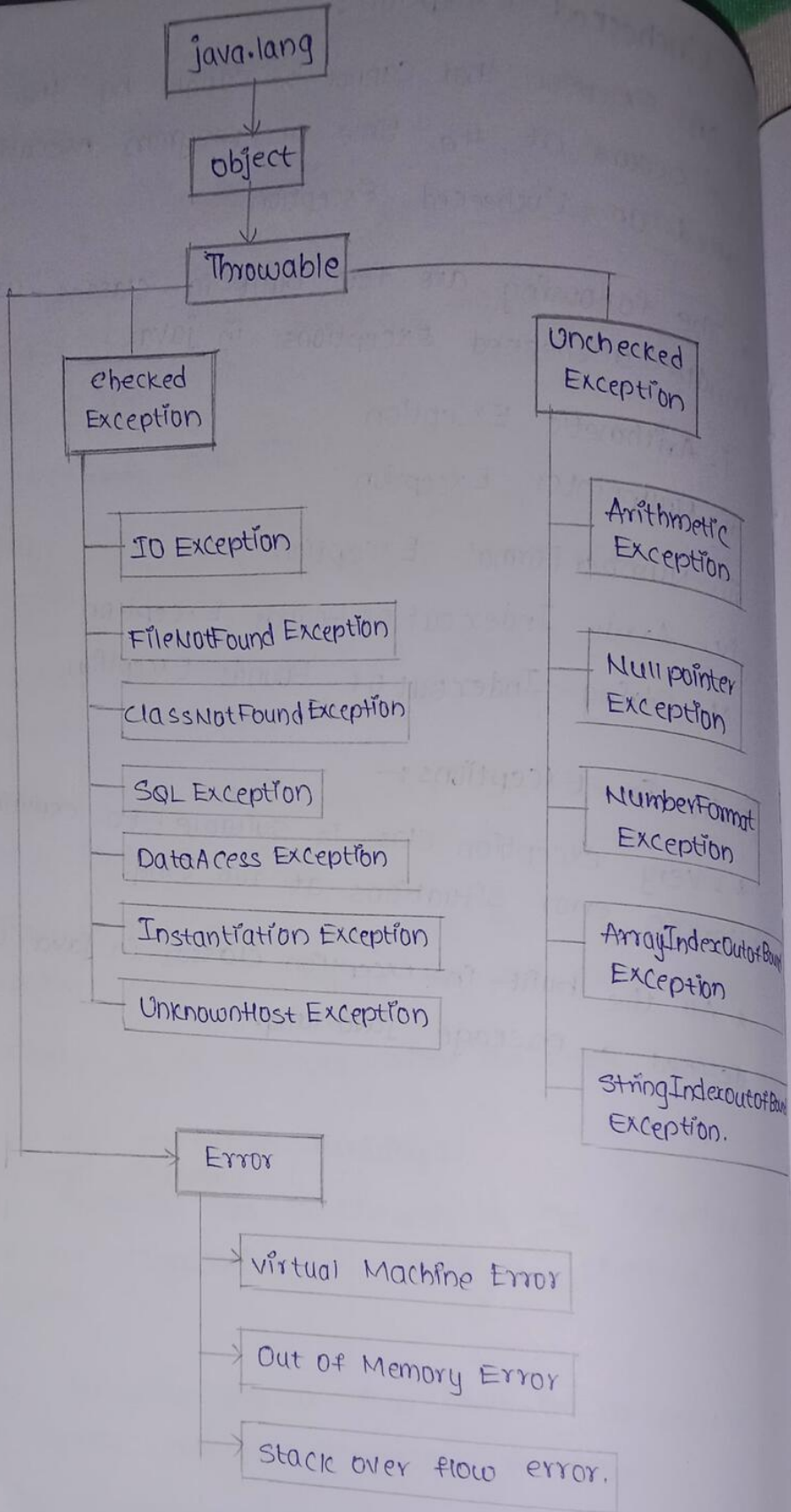
iv. Array Index out of Bounds Exception

v. String Index out of Bounds Exception.

Built-in-Exceptions :-

* Every exception class is suitable to explain certain error situations at run time.

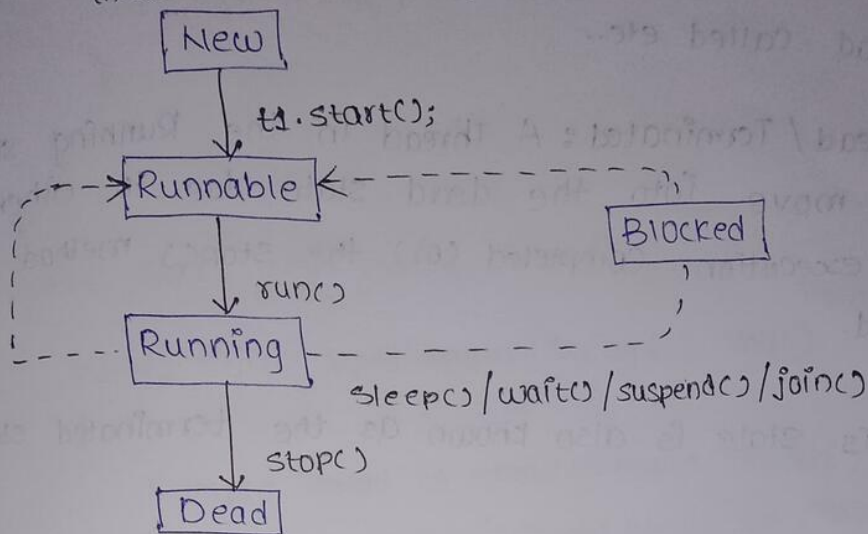
* All the built-in-exception classes in java were defined a package java.lang.



⇒ MULTITHREADED PROGRAMMING:

- * A Thread is a light weight process.
- * A Thread is a subpart of a process that can run individually.
- * Thread Life cycle: The life cycle of a thread is shown in the following figure.

```
Thread t1 = new Thread();
```



1. New/born: When a thread object is created using new, then the thread is said to be in the New state.

* This state is also known as bom state.

```
Thread t1 = new Thread();
```

2. Runnable / Ready: When a thread calls `start()` method, then the thread is said to be in the Runnable state.

* This state is also called as a Ready state

```
t1.start();
```

3. Running: When a thread calls `run()` method, then the thread is said to be Running.

* The `run()` method of a thread called automatically by the `start()` method.

4. Blocked / waiting: A Thread in the Running state may move into the blocked state due to various reasons like `sleep()` method called, and `join()` method called etc..

5. Dead / Terminated: A Thread in the Running state may move into the dead state due to either its execution completed (or) the `stop()` method called.

* This state is also known as the terminated state.

* Creating Threads:

There are mainly two ways to create a thread.

They are:-

> By Extending thread class: To create a thread by using thread class, follow the steps given below.

Step-1: Create a class that extends thread class.

Step-2: Override the `run()` method with the code that is to be executed by the thread.

Step-3: Create the object of the newly created class in the `main()` method.

Step-4: Call the `start()` method on the object created in the above step.

Example:

class Sample Thread extends Thread

{ public void run()

{ System.out.println(" Thread is under Running...");

for(int i=1; k=10; i++)

{ System.out.println(" i=" + i);

}

}

} public class My - Thread - Test {

public static void main(String[] args)

{

Sample Thread t1 = new Sample Thread();

System.out.println(" Thread about to start...");

t1.start();

}

}

Output:

Thread about to start...

Thread is under Running...

i=1

i=2

i=3

i=4

i=5

i=6

i=7

i=8

i=9

i=10

> By implementing Runnable Interface, To create a thread using Runnable Interface, follow the steps given below.

Step-1: Create a class that implements Runnable interface.

Step-2: Override the run() method with the code which is to be executed by the thread.

Step-3: Create the object of the newly created class in the main() method.

Step-4: Call the start() method on the thread class object created in the above step.

* Example :

Class Sample Thread implements Runnable

```
{
    public void run()
    {
        System.out.println("Thread is under Running");
        for(int i=1; i<=10; i++)
        {
            System.out.println("i=" + i);
        }
    }
}
```

Public Class My-Thread-Test2

```
{
    public static void main (String[] args)
    {
        Sample Thread thread object = new SampleThread();
        Thread thread = new Thread (thread object);
        System.out.println("Thread about to start...");
        thread.start();
    }
}
```

Output:-

Thread about to start.....

Thread is under Running.....

i=1

i=2

i=3

i=4

i=5

i=6

i=7

i=8

i=9

i=10.

* Thread Priorities:-

* Priorities in threads is a concept where each thread is having a priority.

* The default priority is set to 5 as expected

i. Minimum priority is set to 1.

ii. Maximum priority is set to 10.

There are three constants are defined in it namely as follows:

1. public static int NORM_PRIORITY

2. public static int MIN_PRIORITY

3. public static int MAX_PRIORITY.

How to get and set priority of a Thread:-

1. public final int get priority() :-

* java.lang.Thread.get priority() method returns priority of given thread.

2. public final void set priority :-

* java.lang.Thread.set priority() method changes the priority of thread to the value new priority.

⇒ STREAM BASED INPUT/OUTPUT :- T

There are mainly two types of stream based Input/output. They are :

> Byte Streams: There are mainly two types of Byte streams. They are :

i. Input Stream:

- * It is an abstract class that defines model of streaming byte input.
- * It implements Closeable interface.
- * It defines methods for performing input function

Such as

i. Reading Bytes

ii. Closing streams

iii. Marking positions in streams.

iv. skipping a head in stream.

v. Finding the number of Bytes in a streams.

ii. Output Streams:

- * It is an abstract class that defines model of streaming byte output.
 - * It implements Closeable and flushable interfaces.
 - * It defines methods for performing output functions
- Such as,

→ writing bytes.

→ Closing streams

→ Flushing streams.

> Character Streams: There are mainly two types of character streams. They are :

- i. Reader Stream: It is an abstract class that defines model of streaming character input.
- * It implements Closeable and Readable interfaces.

ii. Writer Stream:

- * It is an abstract class that defines model of streaming character output.
- * It implements Closeable, Flushable, and Appendable interface.

⇒ STREAM BASED FILES:

There are mainly two types of stream based files.

They are:-

* Reading / Writing Bytes:

- * The subclasses of Reader and Writer implements streams that handle bytes.

- * The Two subclasses used are:

i. FileInputStream - for reading bytes.

ii. FileOutputStream - for writing bytes.

* Reading / writing Characters:

- * The subclasses of Reader and Writer implements streams that handle characters.

- * The two subclasses used are:

i. FileReader - for reading characters

ii. FileWriter - for writing characters.

⇒ UTILITY CLASSES :-

1. Array List: Class provides resizable-array and implements the List interface.
2. HashMap: Class is the Hash table based implementation of the Map interface.
3. HashSet: Class implements the Set interface, backed by a hash table.
4. Priority Queue: Class is an unbounded priority queue based on a priority heap.
5. Random: class interface instance is used to generate a stream of pseudorandom numbers.
6. Scanner: class is a simple text scanner which can parse primitive type and strings using regular expressions.
7. String Tokenizer: Class allows an application to break a string into tokens.
8. Tree Map: Class is the Red-black tree based implementation of the Map interface.

⇒ STRING:

Definition: String basically represents, sequence of char values.

* Creating String Object: There are mainly two ways to create string object. They are:-

> String Literal:

* In Java, string is an object that represents a sequence of characters.

* The java.lang.String class is used to create a string object.

Ex:- String S1 = "Welcome".

> New Keyword:

* The java String is immutable which means it cannot be changed.

* Whenever we change any string, a new instance is created.

Ex: String S2 = new String("Welcome");

* String Class methods:-

i. CharAt(int index): returns char value for the particular index.

ii. length(): returns string length.

iii. Substring(int beginIndex): returns substring for given begin index.

iv. join(): returns a joined string.

v. equals(): Checks the equality of string with the given object.

vi. concat(): concatenates the specified string.

vii. toLowerCase(): returns a string in lowercase.

viii. toUpperCase(): returns a string in uppercase.

UNIT-III

JDBC

- > JDBC Overview
- > JDBC Implementation
- > Connection Class
- > Statements
- > Catching Database Results
- > Handling Database Queries
- > Networking
- > Inet Address class
- > URL class
- > TCP sockets
- > UDP sockets
- > Java Beans
- > RMI.

⇒ JDBC OVERVIEW :-

Definition: JDBC stands for Java Database Connectivity.

It is a Java application programming interface (API) that allows to connect and execute the query with the database.

JDBC Drivers: JDBC Driver is a software component that enables java application to interact with the database.

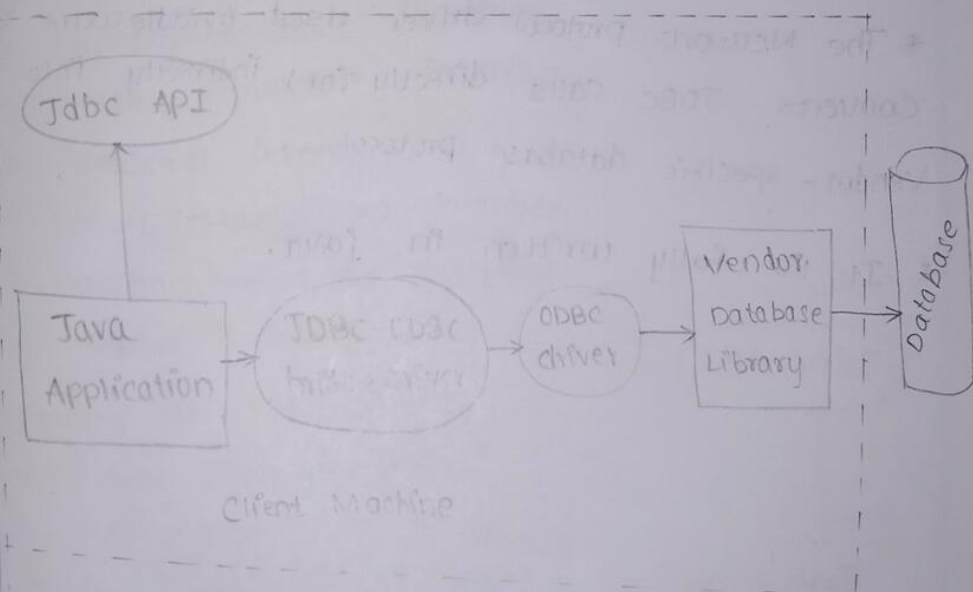
* Types of JDBC Drivers :-

1. JDBC-ODBC Bridge driver :-

* The JDBC-ODBC bridge driver uses ODBC driver to connect to the database.

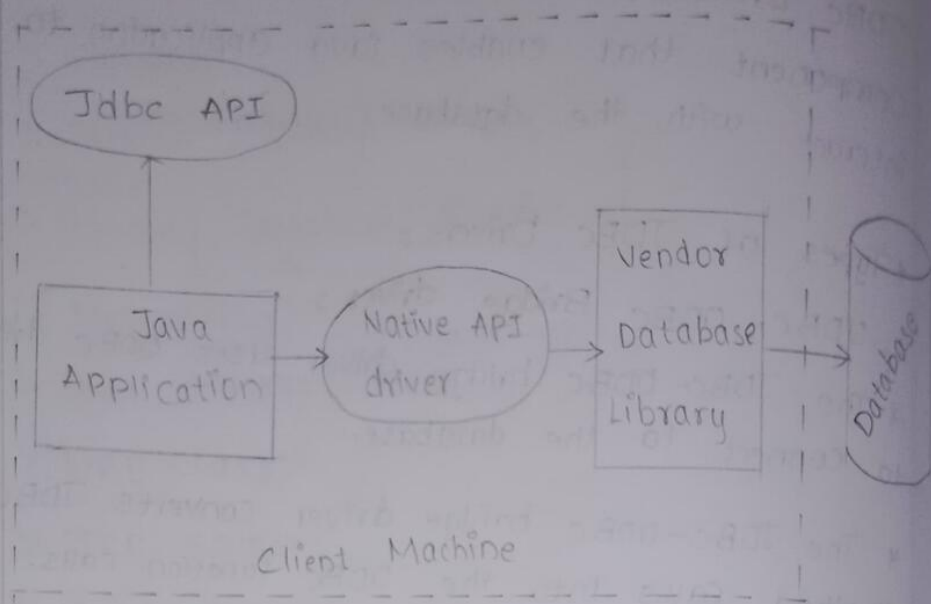
* The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls.

* This is now discouraged because of thin driver.



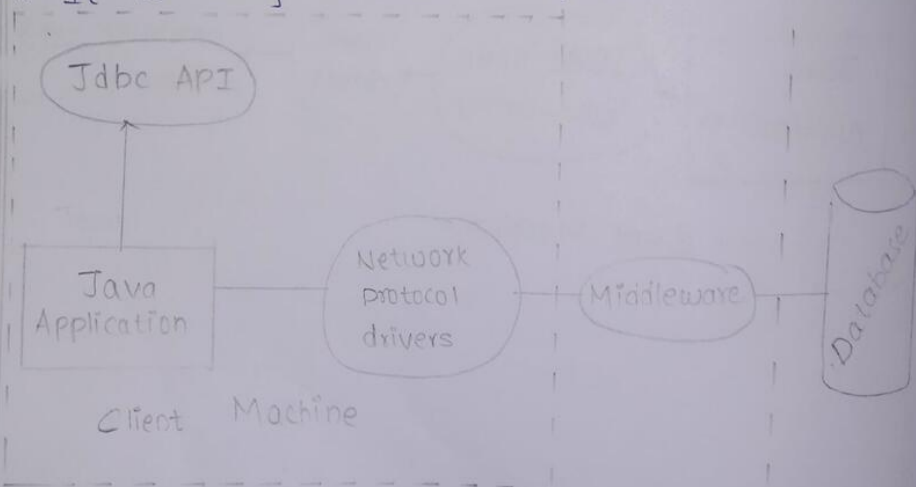
2. Native - API driver :-

- * The Native API driver uses the client-side libraries of the database.
- * The drivers converts JDBC method calls into native calls of the database API.
- * It is not written entirely in Java.



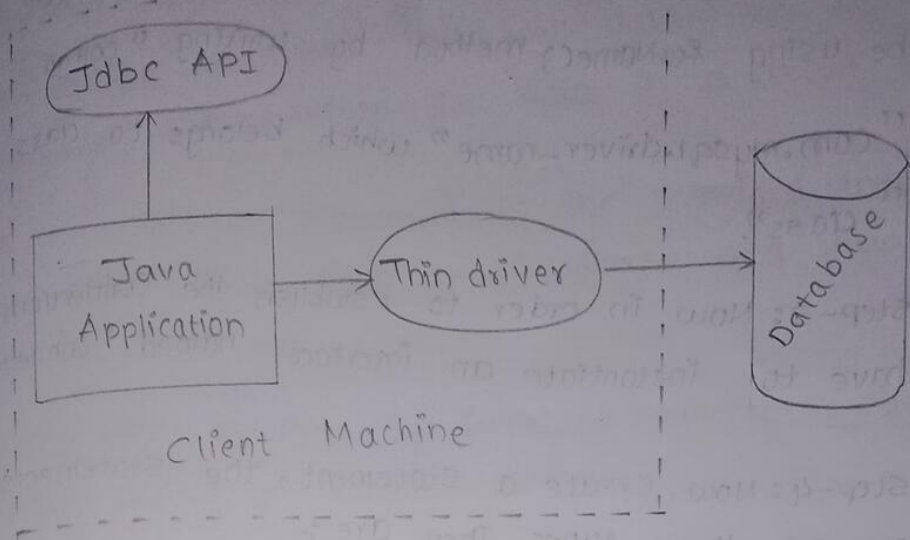
3. Network protocol driver :-

- * The Network protocol driver used middleware that converts JDBC calls directly (or) indirectly into the Vendor-specific database protocol.
- * It is fully written in Java.



4. Thin driver :-

- * The Thin driver converts JDBC calls directly into the vendor-specific database protocol.
- * That is why it is known as thin driver.
- * It is fully written in java language.



Interfaces of JDBC API :-

1. Driver Interface
2. Connection Interface
3. Statement Interface
4. PreparedStatement Interface.
5. Callable Statement Interface.
6. ResultSet Interface.
7. Result Set Meta Data Interface
8. Database Meta Data Interface.
9. Rowset Interface.

Classes of JDBC API :-

1. Driver Manager Class
2. Blob Class.
3. Clob Class
4. Types Class

⇒ JDBC IMPLEMENTATION / CONNECTION CLASS / STATEMENTS :-

Step-1 :- First we need to import the package, for sql database it is namely `"java.sql.*"`,

Step-2 :- Now in order to load the driver, we will be using `forName()` method by writing `"com.mysql.jdbc.Driver"` which belongs to class `"Class"`.

Step-3 :- Now in order to establish the connection, we have to instantiate an interface named `"Connection"`.

Step-4 :- Now, create a statement, the statements are of three types. They are :-

i) Normal Statements: The Normal Statements interface represents the static SQL statements, it helps you to create a general purpose SQL statements using java.

ii) Prepared Statements: Prepared statements represents a precompiled SQL statements, that can be executed many times.

iii) Callable Statements: The Callable statements interface provides methods to execute stored procedures.

Step-5 :- Once the statements is created, now we will be executing the query in our database.

Step-6: Now, process the results.

Step-7: Lastly, closing the connections to release and free up memory resources by closing the object of interface and statement objects.

⇒ CATCHING DATABASE RESULTS:

* The SQL statements that read data from a database query, return the data in a result set.

* The SELECT statement is the standard way to select rows from a database and view them in a result set.

* The java.sql.ResultSet interface represents the result set of a database query.

* The methods of the ~~re~~ ResultSet interface can be broken down into three categories. ~~to~~

They are:

I. Navigational methods: Used to move the cursor around.

II. Get methods: Used to view the data in the columns of the current row being pointed by the cursor.

III. Update the methods: Used to update the data in the columns of the current row.

* JDBC provides the following connection methods to create statements with desired ResultSet.

> CreateStatement(int RS Type, int RS Concurrency);

> PrepareStatement(String SQL, int RS Type, int RS Concurrency);

⇒ HANDLING DATABASE QUERIES :-

1. Establish a connection to your Database:
You can use libraries like "mysql" to connect to different types of database.

2. Write SQL queries: Depending on the type of database you are using, you would write SQL queries.

3. Execute the queries: Using the appropriate functions provided by the database library, you can execute the queries and retrieve the results.

4. Handle the results: Once you receive the results from the database, you can process them further, manipulate the data (or) display it on your web page.

Example: Using Node.js and MySQL. ~~in~~ javascript

```
const mysql = require('mysql');  
// create a connection to the database.
```

```
const connection = mysql.createConnection
```

```
{  
  host: 'localhost',  
  user: 'Username',  
  password: 'password',  
  database: 'mydatabase',  
};
```

```
// Execute a query
```

```
connection.query('SELECT * FROM Users',  
  function(error, results, fields)
```

```
{  
  if (error) throw error;
```

```
// process the results
```

```
console.log(results);
```

```
};
```

```
// close the connection
```

```
connection.end();
```

```
""
```

⇒ NETWORKING :-

definition: Networking is primarily achieved through HTTP (Hypertext Transfer Protocol) is a protocol that defines how client and servers communicate over the internet.

* Concepts of Networking :-

1. HTTP Requests: Clients can send different types of HTTP requests to servers.
2. AJAX: Asynchronous javascript and XML is a technique that allows you to send HTTP requests from a web page without reloading the entire page.
3. Fetch API: The Fetch API is a modern javascript API that provides an interface for making HTTP request.
4. RESTful APIs: REST (Representational State Transfer) is an architectural style that defines a set of principles for building web services.
5. Websocket: Websocket is a communication protocol that provides full-duplex communication between a client and a server over a single, long-lived connection.

⇒ INET ADDRESS CLASS:-

Definition: Java Inet Address class represents a numerical IP address and domain name of the host.

* Methods of InetAddress Class:

> InetAddress getByName(String host):

Returns the instance of InetAddress containing LocalHost IP and name.

> InetAddress getLocalHost():

It returns the instance of InetAddress containing local host name and address.

> String getHostName():-

It returns the host name of the IP address.

> String getHostAddress():-

It returns the IP Address in String format.

Example:

```
import java.io*;
```

```
import java.net*;
```

```
public class Inet Demo  
{
```

```
    public static void main(Strings[] args)
```

```
    {  
        try {
```

```
            InetAddress ip = InetAddress.getByName
```

```
                ("www.google.com");
```

```
            System.out.println("HostName: " + ip.getHostName());
```

```
            System.out.println("IPAddress: " + ip.getHostAddress());
```

```
        }  
    }
```

```
    catch (Exception e)
```

```
    {  
        System.out.println(e);  
    }
```

* Output:

HostName : www.google.com

IP Address : 206.51.231.148.

⇒ URL CLASS:-

Definition: URL stands for Uniform Resource Locator is simply a string of text that identifies all the resources on the internet, telling us the address of the resource, how to communicate with it, and retrieve something from it.

http://www.geeksforgeeks.org/how-to-design-a-tiny-url-or-url-shortener/
↓
Protocol Host Machine File Name

* Components of a URL Class:

1. protocol: HTTP is the protocol here.
2. Hostname: Name of the machine on which the resource lives.
3. FileName: The PathName to the file on the machine.
4. Port Number: Port Number to which to connect (typically optional).

* Constructors of the URL Class :-

1. URL (String address) throws MalformedURLException
Exception: It creates a URL object from the specified String.

2. URL (String protocol, String host, String file) :-
Creates a URL object from the specified protocol, host, and filename.

3. URL (String protocol, String host, int port, String file) :-
Creates a URL object from protocol, host, port, and filename.

4. URL (URLContext, String spec) :- Creates a URL object by parsing the given spec in the given Context.

⇒ TCP Sockets :-

Definition :- TCP stands for Transmission Control protocol, which allows for reliable communication between two applications, TCP is typically used over the Internet protocol, which is pre-referred to as TCP/IP.

* Establishing a TCP Connection :-

* The Server instantiates a ServerSocket object, denoting which port number communication is to occur on.

* The Server invokes the accept() method of the ServerSocket class.

* This method waits until a client connects to the server on the given port.

* After the server is waiting, a client instantiates a socket object, specifying the server name and port number to connect to.

TCP Program:-

server.java:

```
import java.net.*;
import java.util.*;
import java.io.*;

class Server
{
    public static void main (String args[]) throws
    Exception {
        ServerSocket ss = new ServerSocket (5555);
        Random r = new Random();
        while (true)
        {
            Socket s = ss.accept();
            OutputStream os = s.getOutputStream();
            DataOutputStream dos = new DataOutputStream(os);
            dos.writeInt(r.nextInt());
            s.close();
        }
    }
}
```

Client.java:

```
import java.net.*;
```

```
import java.util.*;
```

```
import java.io.*;
```

```
class Client
```

```
{  
    public static void main (String args[]) throws
```

```
Exception
```

```
{
```

```
    Socket S = new Socket("local host", 5555);
```

```
    InputStream is = S.getInputStream();
```

```
    DataInputStream dis = new DataInputStream(is);
```

```
    int i = dis.readInt();
```

```
    System.out.println("random value received is=" +
```

```
        i);
```

```
    S.close();
```

```
}
```

⇒ UDP Sockets:-

Definition: UDP stands for User Datagram Protocol, a Connection-less protocol that allows for packets of data to be transmitted, between applications.

* Classes of UDP:-

1. Datagram Packet:-

* In UDP's terms, data transferred is encapsulated in a unit called datagram.

* You can create a Datagram packet object by using one of the following constructions.

- i. Datagram packet (byte[] buf, int length)
- ii. Datagram packet (byte[] buf, int length),
Inet Address address, int port)

2. Datagram Socket :-

- * You Use Datagram Socket to send and receive Datagram packets.
- * You can create a datagram socket object by using one of the following constructors.
 - i. Datagram Socket()
 - ii. Datagram Socket(int port).

* UDP Program :

Receiver.java :

```
import java.net.*;
```

```
import java.util.*;
```

```
import java.io.*;
```

```
class Receiver
```

```
{  
    public static void main(String args[]) throws
```

```
Exception {
```

```
    Datagram Socket ds = new Datagram Socket (4444);
```

```
    byte buf[] = new byte[60];
```

```
    while(true){
```

```
        Datagram Packet dp = new Datagram Packet  
            (buf, buf.length);
```

```
        ds.receive(dp);
```

```
        String s = new String(dp.getData());
```

```
        System.out.println(s);
```

```
    }
```

```
}
```

```
}
```

Sender.java :-

```
import java.net.*;  
import java.util.*;  
import java.io.*;  
class sender  
{  
    public static void main (String args[]) throws  
        Exception {  
        DatagramSocket ds = new DatagramSocket();  
        InetAddress i = InetAddress.getByName("localhost");  
        byte buf[] = args[0].getBytes();  
        DatagramPacket dp = new DatagramPacket  
            (buf, buf.length, i, 4444);  
        ds.send(dp);  
    }  
}
```

⇒ JAVA BEANS :-

Definition :- A JavaBean is a specially constructed Java class written in the Java and coded according to the JavaBeans API specifications.

* JavaBeans Properties :-

1. `getPropertyNames()`: For example, if property name is `firstName`, your method name would be `getFirstName()` to read that property, this method is called accessor.
2. `setPropertyNames()`: For example, if property name is `firstName`, your method name would be `setFirstName()` to write that property, this method is called mutator.

JavaBeans Example:

Consider a student class with few properties

```
package edu.kmit;
```

```
public class students Bean implements java.io.Serializable
```

```
{  
    private String firstName = null;
```

```
    private String lastName = null;
```

```
    private int age = 0;
```

```
    public students Bean() {}
```

```
    public String getFirstName() {
```

```
        return firstName;
```

```
    }
```

```
    public String getLastName() {
```

```
        return lastName;
```

```
    }
```

```
    public int getAge() {
```

```
        return age;
```

```
    }
```

```
    public void setFirstName(String firstName) {
```

```
        this.firstName = firstName;
```

```
    }
```

```
    public void setLastName(String lastName) {
```

```
        this.lastName = lastName;
```

```
    }
```

```
    public void setAge(Integer age) {
```

```
        this.age = age;
```

```
    }
```

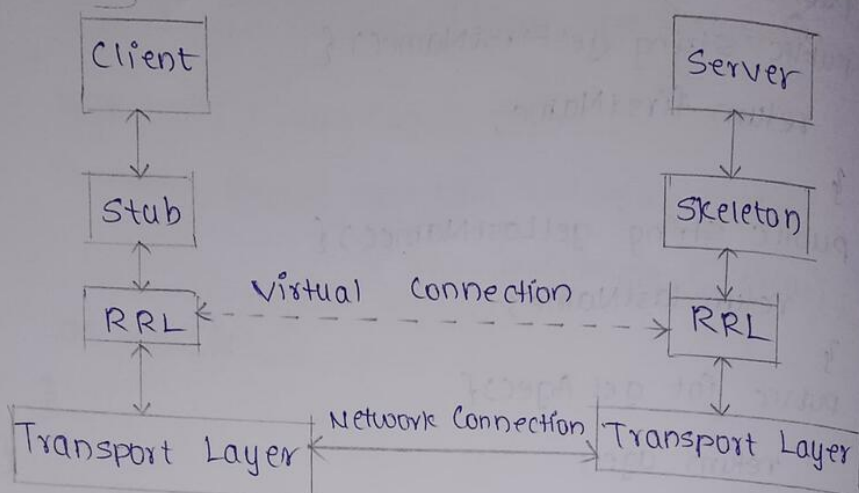
```
}
```

⇒ RMI :

Definition: RMI stands for Remote Method Invocation. It is a mechanism that allows an object residing in one system to access/invoke an object running on another JVM.

* Architecture of an RMI :-

Diagram



* Components of this architecture :

> Transport Layer: This Layer Connects the Client and the Server, it manages the existing connection and also sets up new new connections.

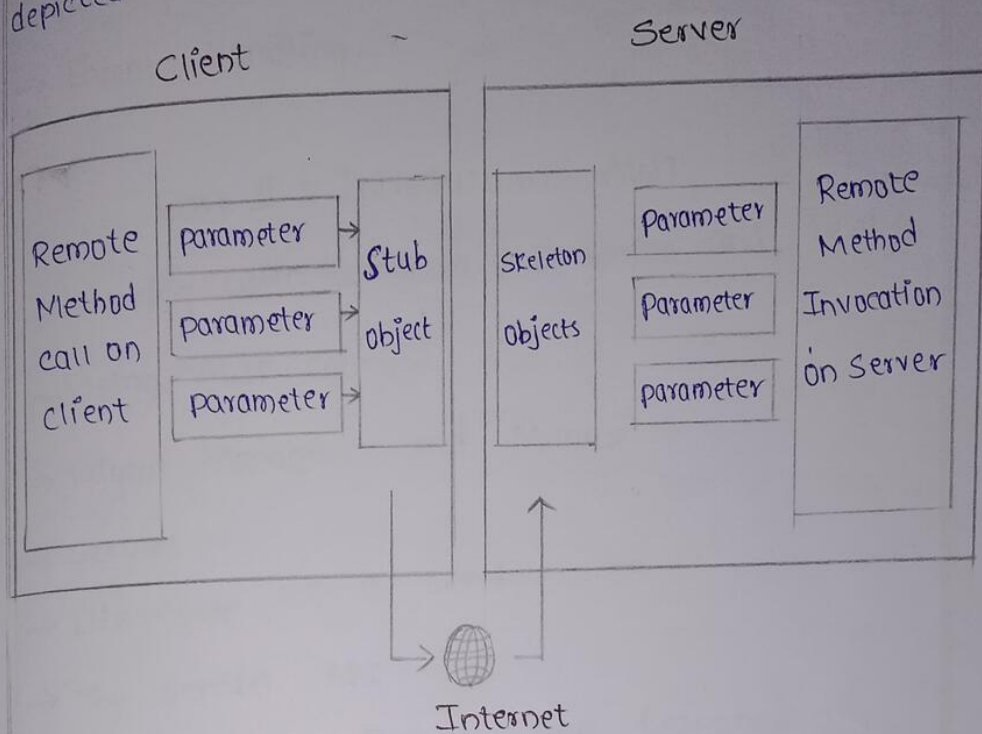
> Stub: A Stub is a representation of the remote object at client, it refer resides in the client system; it acts as a gateway for the client program.

> Skeleton: This is the object which resides on the Server side, Stub Communicates with this skeleton to pass request to the remote object.

> RRL (Remote Reference Layer) :- It is the layer which manages the references made by the client to the remote object.

→ Working of RMI :-

* The Communication between client and server is handled by using two intermediate objects: stub object and skeleton object as also can be depicted from below media as follows:



* These are the steps to be followed sequentially to implement interface as defined below as follows:

step-1: Defining a remote interface.

step-2: Implementing the remote interface.

step-3: Creating stub and skeleton objects from the implementation class using rmic.

Step-4:- Start the rmi registry.

Step-5:- Create and execute the server application program.

Step-6:- Create and execute the client application program.

UNIT - IV

Part I : APPLETs

- Java Applets
- Life cycle of an applet
- Adding images to an applet.
- passing parameters to an applet.
- Adding sound to an applet.
- Event Handling.

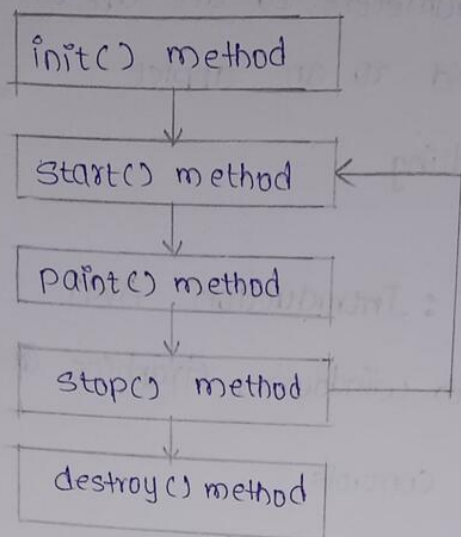
Part II : Introduction AWT

- Working with windows, Graphics and Text.
- Using AWT Controls.
- Layout Managers and Menus.
- Servlet
- Life Cycle of a Servlet
- The Servlet API
- Handling HTTP Request and Response
- Using Cookies
- Session Tracking
- Introduction to JSP

Part-I: APPLETs

⇒ JAVA APPLET:- Java Applet is a special type of program that is embedded in the webpage to generate the dynamic content. It runs inside the browser and works at client side.

⇒ *LIFE CYCLE OF AN APPLET:-



- i. `init()` method: It is used to initialize the Applet. It is invoked only once.
- ii. `start()` method: It is used to start the Applet. It is invoked after the `init()` method (or) browser is maximized.
- iii. `paint()` method: It is used to paint the Applet. It provides Graphics class object that can be used for drawing oval, rectangle, arc etc. ...
- iv. `stop()` method: It is used to stop the Applet. It is invoked when Applet is stop (or) browser is minimized.
- v. `destroy()` method: It is used to destroy the Applet. It is invoked only once.

ADDING IMAGES TO AN APPLET :-

```
import java.awt.*;  
import java.applet.*;  
/* <applet code = "App005" height = 600  
width = 600> </applet>* /
```

```
public class App005 extends Applet  
{  
    image picture;  
    public void init()  
    {  
        picture = getImage(getDocumentBase(). "img1.jpg");  
    }  
    public void paint(Graphics g)  
    {  
        g.drawImage(picture, 30, 30, this);  
    }  
}
```

=> ADDING SOUND TO AN APPLET :-

```
import java.applet.*;  
import java.awt.*;  
import java.awt.event.*;  
/* <applet code = "App006" height = 600 width = 600>  
    </applet>* /
```

```
Public class App006 extends Applet implements  
    ActionListener
```

```
{  
    Button b1, b2;  
    AudioClip audioclip;  
    Public void init()  
    {  
        b1 = new Button("play");  
        add(b1);  
        b2 =
```

```

        b1.add ActionListener (this);
        b2 = new Button("stop");
        add (b2);
        b2.add ActionListener (this);
        AudioClip = get AudioClip (get codeBase(), "audio.wav");
    }

    public void action performed (Action Event ae)
    {
        Button Source = (Button)ae.getSource();
        if (Source.getLabel().equals("play"))
        {
            AudioClip.play();
        }
        else if (Source.getLabel().equals("stop"))
        {
            AudioClip.stop();
        }
    }
}

```

⇒ PASSING PARAMETERS TO AN APPLET:-

```

import java.applet.Applet;
import java.awt.Graphics;

/* <applet Code = "App004" width = 400 height = 400>
<param name = "Username" value = "shiva">
    </applet>*/

public class APP004 extends Applet
{
    String s;
    public void init()
    {
        s = get Parameter ("Username");
    }
}

```

```
public void paint(Graphics g)
```

```
{  
    g.drawString("S, 100, 100);
```

```
}
```

```
}
```

⇒ EVENT HANDLING :-

* Event: Changing the state of an object is known as an event.

Ex: click on button, dragging mouse etc...

* Event Handling: Java uses event delegation model for handling / processing the event.

* Terminology in event handling:

> Source: Like button, checkbox, applet etc... on which user has generated the event.

> Type of Event: Example if the user clicks on a button, the type of event is called as the action event. If the user clicks on the applet, it is mouse event.

> Listener: The class(s) who wants to know about a particular event and respond.

> Event Registration: The classes who are interested in knowing and processing any type of event on any source must register for the same.

* Event Delegation Model: When an event is generated on the source, JRE will create an object of corresponding Event class with the information about generate event and delegates to the registered listeners.

Now the listener class will invoke the corresponding method which processes the event.

* Event classes and Listener interfaces:

ActionEvent

ActionListener

MouseEvent

MouseListener and MouseListener

KeyEvent

KeyListener

ItemEvent

ItemListener

TextEvent

TextListener

AdjustmentEvent

AdjustmentListener

WindowEvent

WindowListener

* Handling Mouse Events - Example :-

//making use of inner classes

```
import java.applet.*;
```

```
import java.awt.*;
```

```
import java.awt.event.*;
```

```
// <applet code = "a9" width = 400 height = 400>
```

```
</applet>
```

```
public class a9 extends Applet
```

```
{
```

```
    public void init()
```

```
    {
```

```
        add MouseListener(new MouseAdapter()
```

```
        {
```

```
            public void mouseClicked(MouseEvent me)
```

```
            {
```

```
                show status ("mouse clicked");
```

```
            }
```

```
        }
```

```
    };
```

```
}
```

```
}
```

* Handling Keyword Events - Example :-

```
import java.applet.*;  
import java.awt.*;  
import java.awt.event.*;
```

```
//<applet code = "a6" width = 400 height = 400 </applet>  
public class a6 extends Applet implements  
KeyListener {
```

```
{ int x = 30, y = 30;
```

```
String S = "";
```

```
public void init() {
```

```
addKeyListener(this);
```

```
} public void keyTyped(KeyEvent ke)
```

```
{ S += ke.getKeyChar();
```

```
repaint();
```

```
} public void keyPressed(KeyEvent ke)
```

```
{ showStatus("Key pressed");
```

```
} public void keyReleased(KeyEvent ke)
```

```
{ showStatus("Key Released");
```

```
} public void paint(Graphics g)
```

```
{ g.drawString(S, x, y);
```

```
}
```

```
}
```

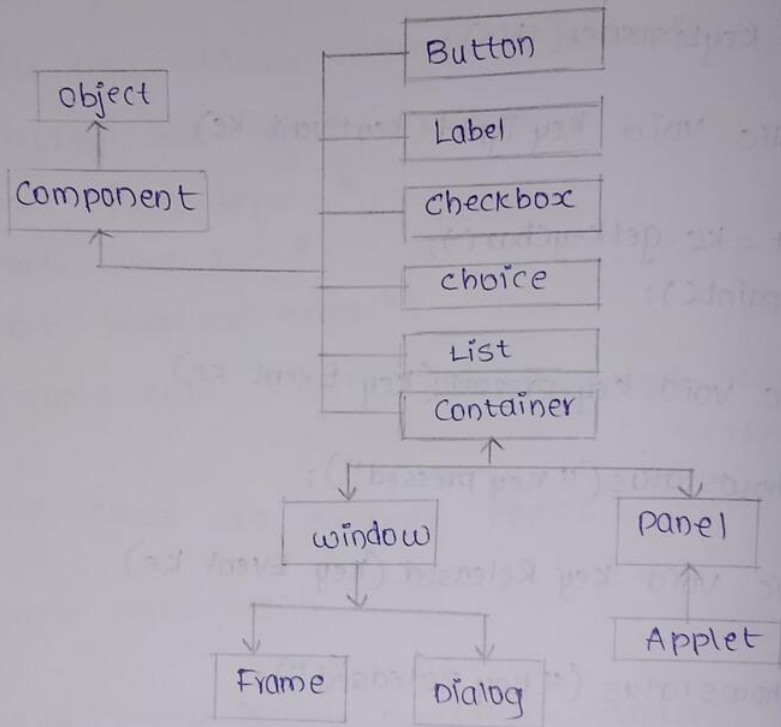
Part - II Introduction

AWT

⇒ **JAVA AWT DEFINITION:** Java AWT [Abstract Window Toolkit] is an API to develop GUI window-based applications in Java.

⇒ WORKING WITH WINDOWS, GRAPHICS AND TEXT:-

* The java.awt package provides classes for AWT API such as TextField, Label, TextArea, RadioButton, CheckBox, choice, List etc...



* **Container:** The Container is a Component in AWT that can contain other components like buttons, textfields, labels etc...

* **Window:** The Window is the Container that has no borders and menu bars. You must use frame, dialog (or) another window for creating a window.

* **Panel:** The Panel is the Container that doesn't contain title bar and menu bars. It can have other components like button, textfield etc...

* Frame: The Frame is the container that contain title bar and can have menu bars. It can have other components like button, textfield etc...

⇒ USING AWT CONTROLS:-

- i. Labels: It is used to display a single line of read only text.
- ii. push Button: Contains a label and generates an event when presses.
- iii. Check Box: contains a label and used to turn an option on (or) off. It consists of a small box with a check mark (or) not.
- iv. Choice List: It creates a popup menu where user chooses.
- v. Lists: Multiple choice and scrolling selection list.
- vi. Scroll Bars: Scroll bars are used to select continuous values between a specified minimum and maximum. Scroll bars may be oriented horizontally (or) vertically.
- vii. TextField: Single line text entry.
- viii. Text Area: Multiline text editor.

⇒ LAYOUT MANAGERS AND MENU'S :

- i. Border Layout: It is used to arrange the components in five regions: north, south, east, west and centre. Each region may contain one component only.

Ex:-

```
Container - component.add(Component, BorderLayout.NORTH);
```

2. Grid Layout :- It is used to arrange the components in rectangular grid.

Ex :

```
Container_component.setLayout(new GridLayout(3,3));
```

```
Container_component.add(Component-1);
```

3. Flow Layout :- It is used to arrange the components in a line, one after another. It is the default layout of applet (or) panel.

Ex :

```
Container_component.setLayout(new FlowLayout  
(FlowLayout.RIGHT));
```

```
Container_component.add(Component-1);
```

4. Card Layout :- class manages the components in such a manner that only one component is visible at a time.

Ex :-

```
c = getContentPane();
```

```
Card = new CardLayout(40, 30);
```

```
// create CardLayout object with 40 hor space and  
30 ver space. setLayout(Card);
```

```
b1 = new JButton("Apple");
```

```
b2 = new JButton("Boy");
```

```
c.add("a", b1);
```

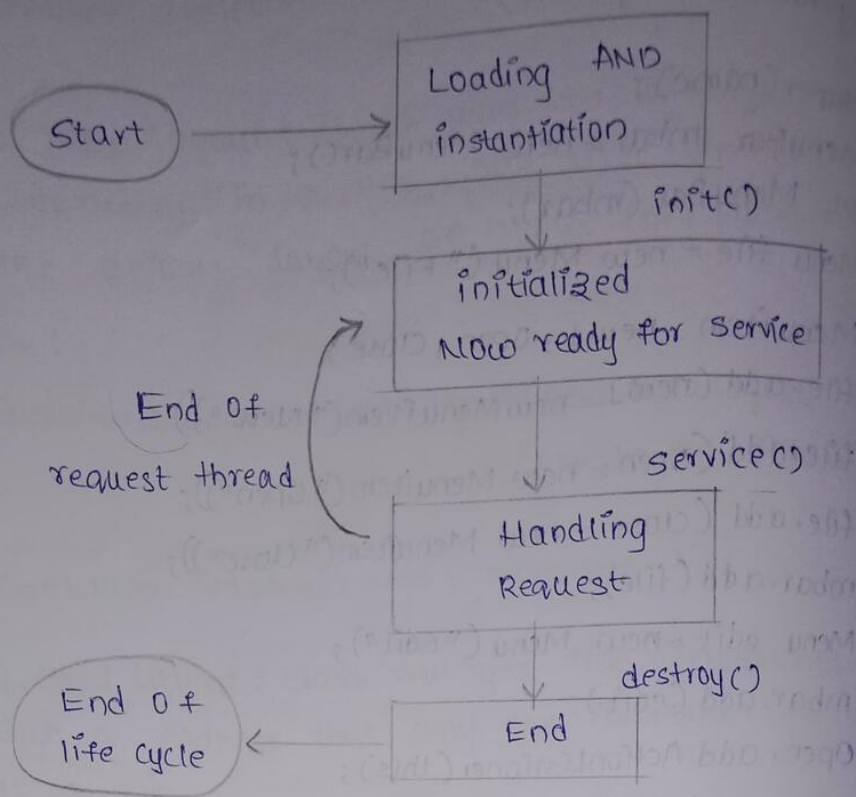
```
c.add("b", b2);
```

* Menus - Example:
class myFrame extends Frame implements
ActionListener

```
{ String msg = "";  
  myFrame(String name)  
  {  
    super(name);  
    MenuBar mbar = new MenuBar();  
    setMenuBar(mbar);  
    Menu file = new Menu("File");  
    MenuItem new1, open, close;  
    file.add(new1 = new MenuItem("New"));  
    file.add(open = new MenuItem("Open"));  
    file.add(close = new MenuItem("Close"));  
    mbar.add(file);  
    Menu edit = new Menu("Edit");  
    mbar.add(edit);  
    open.addActionListener(this);  
    close.addActionListener(this);  
    public void actionPerformed(ActionEvent ae)  
    {  
      msg = (String) ae.getActionCommand();  
      repaint();  
    }  
    public void paint(Graphics g)  
    {  
      g.drawString(msg, 100, 100);  
    }  
  }  
}
```

=> SERVLET DEFINITION:- Servlet Technology is used to create a web application (resides at Server side and generates a dynamic web page).

=> LIFE CYCLE OF A SERVLET:-



1. Servlet Class is loaded: The Classloader is responsible to load the Servlet class. The Servlet class is loaded when the first request for the Servlet is received by the web Container.

2. Servlet instance is created: The web Container creates the instance of a Servlet after loading the Servlet class. The Servlet instance is created only once in the Servlet life cycle.

3. `init()` method is invoked: The Web Container calls the `init` method only once after creating the Servlet instance. The `init` method is used to initialize the Servlet.

4. `service()` method is invoked: The web container calls the service method each time when request for the servlet is received.

5. `destroy()` method is invoked: The web container calls the destroy method before removing the servlet instance from the service.

⇒ THE SERVLET API:-

Interfaces	Description
1. Servlet	Declares life cycle methods that all servlets must implement.
2. Servlet Config	Allows servlet to get initialization parameters.
3. Servlet Request	Provides client request information to a servlet.
4. Servlet Response	Assist a servlet in sending a response to the client.

Classes	Description
1. Generic Servlet	Provides a basic implementation of the servlet interface for protocol independent servlets.
2. ServletInputStream	Provides an input stream for reading binary data from a client request.
3. ServletOutputStream	Provides an output stream for sending binary data to the client.
4. ServletException	Defines a general exception, a servlet can throw when it encounters difficulty.

* Interfaces and classes from java.servlet.
http package.

* Important interfaces in javax.servlet.http package

> Http Servlet Request.

> Http Servlet Response

> Http Session.

* Important classes in javax.servlet.http Package

> HttpServlet

> Cookie.

⇒ HANDLING HTTP REQUESTS AND RESPONSE:-

* The HTTP servlet class provides specialized methods that handle the various type of HTTP request.

* A Servlet developer typically overrides one of these methods: doDelete(), doGet(), doHead(), doOptions(), doPost(), doPut(), and doTrace().

Example:

// Simple Calculator Example

```
import java.io.*;
```

```
import javax.servlet.*;
```

```
import javax.servlet.http.*;
```

```
public class serv6 extends HttpServlet
```

```
{  
    protected void doGet (HttpServletRequest request,  
                           HttpServletResponse response)
```

```
throws ServletException, IOException
```

```
{  
    int x = Integer.parseInt(request.getParameter("t1"));  
    int y = Integer.parseInt(request.getParameter("t2"));
```

```

String opr = request.getParameter("s1");
opr = opr.trim();
int op = Integer.parseInt(opr);
int result = 0;
switch (op)

```

```

{
    case 1 : result = x + y;
    break;

```

```

    case 2 : result = x - y;
    break;

```

```

    case 3 : result = x * y;
    break;

```

```

    case 4 : result = x / y;
    break;
}

```

```

PrintWriter pw = response.getWriter();

```

```

pw.println("Result = " + result);
}
}

```

⇒ USING COOKIES:

* Cookie is a bit of information created by server for the first client request and will sent back to the client browser.

* Creating & sending cookies to client:

```

Cookie ck1 = new Cookie("username", "pavani");
response.addCookie(ck1);

```

* Accessing all cookies coming from client:

```

PrintWriter out = response.getWriter();

```

```

Cookie ck[] = request.getCookies();

```

```

for (int i = 0; i < ck.length; i++)

```

```

{
    out.print(ck[i].getName() + " ");

```

```

    out.print(ck[i].getValue());
}
}

```

⇒ SESSION TRACKING :-

* Session simply means a particular interval of time.

* Session Tracking is a way to maintain state (data) of user. It is also known as session management in servlet.

* Creating an instance of current date and time and storing in session :-

```
Date d1 = new Date();
```

```
HttpSession s1 = request.getSession();
```

```
s1.setAttribute("mydate", d1);
```

* Accessing the data (date object) from session :-

```
HttpSession = request.getSession(false);
```

```
Date d = (Date)s.getAttribute("mydate");
```

```
response.getWriter().println(d);
```

⇒ INTRODUCTION TO JSP :-

JSP DEFINITION: JSP stands for Java Server Pages, it is a server-side technology which is used for creating web applications, it is used to create dynamic web content.

* Features of JSP :-

> Coding in Jsp is easy: As it is just adding JAVA code to HTML/XML.

> Reduction in the length of code: In Jsp we use action tags, custom tags etc...

> Connection to Database is easier: It is easier to connect website to database and allows to read (or) write data easily to the database.

> Make Interactive websites: In this we can create dynamic web pages which helps users to interact in real time environment.

* JSP Elements:

1. Expression: We can use this tag to output any data on the generated page. These data are automatically converted to string and printed on the output stream.

Syntax:

JSP Expressions are: `<%="Anything"%>`

2. Scriptlets: In this tag we can insert any amount of valid java code and these codes are placed in the jsp service method by the JSP engine.

Syntax: `<%// java codes %>`

3. Directives: A JSP "directive" starts with `<%@` characters. In the directives, we can import packages, and define error handling pages (or) the session information of the JSP page.

Syntax: `<%@ directive attribute = "value"%>`

4. Declarations: This tag is used for defining the functions and variables to be used in the JSP.

Syntax:

`<%!`

`// java codes`

`%>`

→ * Run a simple JSP Page :-

Step-1: Save the JSP file using ".jsp" extension (ex- "hello.jsp")

Step-2: start the server.

Step-3: place your application inside a folder.

Step-4: To execute the JSP script, simply start tomcat server and use a browser to browse an URL of the JSP page i.e.,

http://localhost:port number/your Application Context Root/jspfile then you will see the JSP file is being compiled.

Example of a Jsp web page:

```
<HTML>
```

```
<HEAD>
```

```
<TITLE> A Web Page </TITLE>
```

```
</HEAD>
```

```
<BODY>
```

```
<% out.println("Hello there!"); %>
```

```
</BODY>
```

```
</HTML>
```

UNIT - V

XML AND WEB SERVICES.

- XML
- Introduction - Form Navigation
- XML documents
- XSL
- XSLT
- Web services
- UDDI
- WSDL
- Java web services
- Web resources.

* XML DEFINITION:-

- * XML stands for extensible Markup Language.
- * XML is a markup language much like HTML.
- * XML was designed to store and transport data.
- * XML was designed to be self-descriptive.
- * XML is a W3C Recommendation.

* Applications of XML:-

> Data transfer: You can use XML to transfer data between two systems that store the same data in different formats.

> Web applications: XML gives structure to the data that you can see on webpages. Other website technologies, like HTML, work with XML to present consistent and relevant data to website visitors.

> Documentation: You can use XML to specify the structural information of any technical document. Other programs then process the document structure to present it flexibly.

> Data type: Many programming languages support XML as a data type. With this support you can easily write programs in other languages that work directly with XML files.

=> FORM NAVIGATION:-

* Form Navigation refers to the process of guiding users through different forms on a website.

* It involves creating a logical flow and providing intuitive controls for users to move between forms and complete their tasks.

- * One using Common approach to form navigation is buttons (or) links to switch between forms.
- * Another approach is using tabs (or) a tabbed interface to display different sections of a form.
- * Additionally, form navigation can involve dynamically updating the form based on user input.
- * The key is to design form navigation that is clear, intuitive and efficient, ensuring that users can easily navigate through the forms without confusion (or) frustration.

⇒ XML DOCUMENT :-

- * An XML document is a basic unit of XML information composed of elements and other markup in an orderly package.
- * An XML document can contains wide variety of data.

Example: Books.xml

```
<?xml version="1.0" encoding="UTF-8"?><book store>
<book store>
<book category="cooking">
<title lang="en">Everyday Italian</title>
<author>Giada De Laurentiis</author>
<year>2005</year>
<price>30.00</price>
</book>
<book category="children">
<title lang="en">Harry Potter</title>
<author>JK. Rowling</author>
<year>2005</year>
<price>29.99</price>
```

</book>

<book category = "web">

<title lang = "en" Learning XML </title>

<author> Erik T. Ray </author>

<year> 2003 </year>

<price> 39.95 </price>

</book>

</bookstore>

⇒ XSL :

* Before learning XSLT, we should first understand XSL which stands for Extensible Stylesheet Language.

* It is similar to XML as CSS is to HTML.

* Need for XSL :

* In case of HTML document, tags are predefined such as table, div, and span; and the browser knows how to add style to them and display those using CSS styles.

* But in case of XML documents, tags are not predefined.

* In order to understand and style an XML document, World Wide Web Consortium (W3C) developed XSL which can act as XML based stylesheet Language.

* An XSL document specifies how a browser should render an XML document.

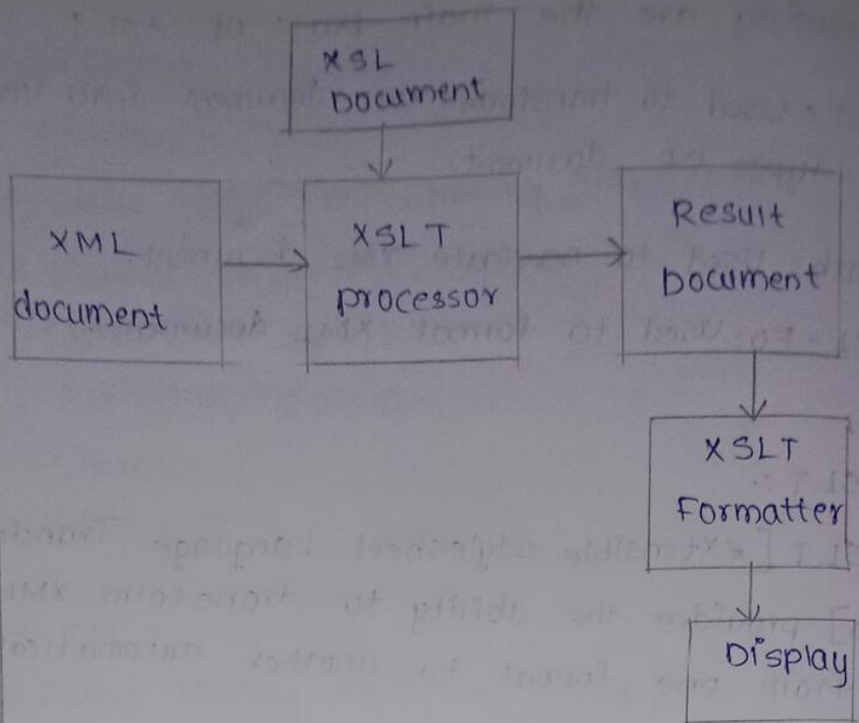
- Following are the main parts of XSL:
1. XSLT: Used to transform XML document into various other types of document.
 2. XPath: Used to navigate XML document.
 3. XSL-FO: Used to format XML document.

⇒ XSLT:-

XSLT [Extensible Stylesheet Language Transformations] provides the ability to transform XML data from one format to another automatically.

XSLT Working:

- An XSLT Stylesheet is used to define the transformation rules to be applied on the target XML document.
- XSLT Stylesheet is written in XML format.
- XSLT processor takes the XSLT Stylesheet and applies the transformation rules on the target XML document and then it generates a formatted document in the form of XML, HTML (or) text format.
- This formatted document is then utilized by XSLT formatter to generate the actual output which is to be displayed to the end-user.



⇒ WEB SERVICES:

* A Web Service is any piece of software that makes itself available over the internet and uses a standardized XML messaging system.

* XML is used to encode all communications to a web service.

Example:

* A Client invokes a web service by sending an XML message, then waits for a corresponding XML response.

* As all communication is in XML, web services are not tied to any one operating system (or) programming language. Java can talk with Perl.

* Windows applications can talk with Unix applications.

- ⇒ UDDI :-
- * UDDI stands for Universal Description Discovery and Integration.
 - * UDDI is a directory for storing information about web services.
 - * UDDI is a directory of web services interfaces described by WSDL.
 - * UDDI Communicates via SOAP.

UDDI Based on:

- * UDDI users world wide web Consortium (W3C)
- * UDDI and Internet engineering Task force (IETF)
- * Internet standards such as XML, HTTP and DNS protocols.

- * UDDI uses WSDL to describe interfaces to web services.

UDDI Benefits:

- * Any industry (or) businesses of all sizes can benefit from UDDI.
- * Before UDDI, there was no Internet standard for businesses to reach their customers and partners with information about their products and services.

* Problems the UDDI specification can help to solve:-

* Making it possible to discover the right business from the millions currently online.

* Defining how to enable commerce once the preferred business is discovered.

How can UDDI be used:-

* Travel agencies could then search the UDDI directory to find the airline's reservation interface (i.e. WSDL file)

* When the interface is found, the travel agency can communicate with the service using SOAP and WSDL.

⇒ WSDL:-

* WSDL stands for Web Services Description Language.

* WSDL is used to describe web services.

* WSDL is written in XML.

* WSDL is a W3C recommendation from 26 June 2007.

→ * WSDL Documents:-

* An WSDL document describes a web service.

* It specifies the location of the service, and the methods of the service, using these major elements.

* Elements of WSDL document:

1. **Types**: Defines the (XML schema) for user defined data types used by the web service.
2. **Message**: Message defines a one-way request (or) response message consist of one (or) more parts.
3. **portType**: Defines the operations that can be performed, and the messages that are involved.
4. **binding**: Binding provides details on how a port Type operation is transmitted over the wire.

* **Service**: defines a collection of end points in terms of nested port elements.

The main structure of a wsdL document looks like this :-

<definitions>

<types>

data type definitions.....

</types>

<message>

definition of the data being

Communicated.....

</message>

<portType>

Set of Operations.....

</PortType>

<binding>

Protocol and data format

specification

</binding>

</definitions>

* WSDL Example :-

<message name = "getTermRequest">

<part name = "term"

type = "xs:string"/>

</message>

<message name = "getTermResponse">

<part name = "value"

type = "xs:string"/>

</message>

<portType name = "glossaryTerms">

<operation name = "getTerm">

<input

message = "getTermRequest"/>

<output

message = "getTermResponse"/>

</operation>

</portType>

⇒ JAVA WEB SERVICES:-

1. The Services that are accessible across various networks are Java web services.
2. Since Java EE6, it has two APIs defined by Java for creating web services Applications.

1. JAX-WS for SOAP Web Services:-

* The Jakarta EE API, known as JAX-WS, is used to develop and create web services, especially for SOAP Service users.

* There are mainly two ways to write JAX-WS application code. They are:-

- i. RPC style
- ii. Document style.

2. JAX-RS for RESTful web services:

1. JAX-RS is a framework for building RESTful web applications.

2. There are mainly two implementations for building JAX-RS. They are:-

- i. Jersey
- ii. RESTeasy

* Implementation of Java web services:

1. Implementing SOAP Web Services with

JAX-WS:-

* First, you need to define service end point interfaces (SEI) which specify, the methods to expose as web service.

* Next, you need to implement SEI with a Java class.

* Then you need to Annotate the SEI and its implementation class with JAX-WS annotations to specify the web service.

* Package web service classes to the war file and deploy it to a web server.

2. Implementing RESTful web services with

JAX-RS.

* First you need to define the resources that represents the web services and its methods.

* Then you need to annotate the resource class and its methods with JAX-RS annotations to specify the web service package.

* At last we need to package the web service classes to the war file and deploy it to a web server.

=> WEB RESOURCES:-

* Web Resources are the resources which are required for proper rendering in the web application, it includes images, script files, and any user-created component libraries.

* The following are the some important points to store the resources.

i. It must be stored in a subdirectory of a resources directory at the web application root: resources / resource-identifier.

ii. A resource packaged in the web applications classpath must be in a subdirectory of the META-INF / resources directory within a web application: META-INF / resources / resource-identifier.

iii. You can use this file structure to package resources in a JAR file bundled in the web application.